

150 Exercícios Práticos
Do Básico ao Avançado

Fundamentos · Arrays · Objetos · Funções · Strings · Números · Controle
de Fluxo

Índice

■ Fácil (exercícios 1–50)

Fundamentos, strings, números, arrays, objetos, controle de fluxo

■ Médio (exercícios 51–100)

map/filter/reduce, closures, objetos avançados, algoritmos

■ Difícil (exercícios 101–150)

Closures avançados, composição funcional, algoritmos complexos

Como usar este caderno

- Resolva cada exercício antes de ver o exemplo de solução.
- Os exemplos mostram uma possível solução — existem muitas formas corretas.
- Use as dicas (■) quando estiver travado, não antes de tentar.
- Os exercícios dentro de cada nível são ordenados por complexidade crescente.
- Exercícios difíceis combinam múltiplos conceitos — é normal levar mais tempo.

■ Exercícios Fácil

1 Olá, Mundo!

Fácil · Fundamentos

Crie um arquivo JS e use o console para exibir a mensagem 'Olá, Mundo!'.

```
console.log('Olá, Mundo!');
```

■ Use `console.log()` — é a função mais básica de saída em JS.

■ console · string

2 Declarando variáveis

Fácil · Fundamentos

Declare três variáveis usando `var`, `let` e `const` com valores à sua escolha. Exiba cada uma no console.

```
var nome = 'Ana';  
let idade = 25;  
const pi = 3.14;  
console.log(nome, idade, pi);
```

■ Prefira `const` por padrão; use `let` quando precisar reatribuir.

■ var · let · const

3 Template literals avançados

Fácil · Strings

Dados `nome`, `profissão` e `cidade` em variáveis, monte a frase 'Sou [nome], trabalho como [profissão] em [cidade].' usando template literals. Depois, exiba a frase em maiúsculas e mostre seu comprimento.

```
const nome = 'Clara', prof = 'dev', cidade = 'Lisboa';  
const frase = `Sou ${nome}, trabalho como ${prof} em ${cidade}.`;   
console.log(frase.toUpperCase());  
console.log(frase.length);
```

■ Template literals com backtick permitem expressões dentro de `${}`.

■ template literals · toUpperCase · length

4 Operadores aritméticos combinados

Fácil · Fundamentos

Dado o preço unitário de um produto e a quantidade comprada, calcule: subtotal, desconto de 10%, total com desconto e total arredondado para 2 casas decimais.

```
const preco = 49.9, qtd = 3;  
const subtotal = preco * qtd;  
const desconto = subtotal * 0.1;  
const total = subtotal - desconto;  
console.log(total.toFixed(2)); // 134.73
```

■ `toFixed()` retorna string; use `parseFloat()` para converter de volta se precisar operar.

■ operadores · toFixed · number

5 Comparações e coerção

Fácil · Fundamentos

Compare os seguintes pares com `==` e `===` e exiba uma mensagem explicando o resultado: `(0, false)`, `('1', 1)`, `(null, undefined)`, `([], false)`. Comente o porquê de cada resultado.

```
console.log(0 == false); // true (coerção)
console.log(0 === false); // false
console.log('1' == 1); // true
console.log(null == undefined); // true
console.log([] == false); // true
```

■ Use sempre `===` para evitar surpresas. `null == undefined` é verdadeiro, mas `null === undefined` é falso.

■ `==` · `===` · coerção · tipos

6 Operador ternário encadeado

Fácil · Fundamentos

Dada uma variável temperatura (número), use o operador ternário para classificar: abaixo de 10 → 'Frio', entre 10 e 25 → 'Agradável', acima de 25 → 'Quente'.

```
const temp = 18;
const clima = temp < 10 ? 'Frio'
  : temp <= 25 ? 'Agradável'
  : 'Quente';
console.log(clima); // Agradável
```

■ Ternários encadeados funcionam, mas para 3+ condições prefira `if/else` ou `switch`.

■ ternário · condição · string

7 Switch com classificação de notas

Fácil · Controle de Fluxo

Dado um valor de nota (0-10), use `switch/case` com faixas para exibir o conceito: 9-10 → 'A', 7-8 → 'B', 5-6 → 'C', abaixo de 5 → 'D'. Dica: use `switch(true)`.

```
const nota = 8;
switch(true) {
  case nota >= 9: console.log('A'); break;
  case nota >= 7: console.log('B'); break;
  case nota >= 5: console.log('C'); break;
  default: console.log('D');
}
```

■ `switch(true)` avalia cada `case` como expressão booleana — truque útil para faixas.

■ `switch` · boolean · faixas

8 Métodos de string encadeados

Fácil · Strings

Dada a string ' javascript é incrível! ', encadeie trim(), split(' '), map para capitalizar cada palavra e join(' '). Exiba o resultado final.

```
const s = ' javascript é incrível! ';
const resultado = s.trim()
  .split(' ')
  .map(p => p[0].toUpperCase() + p.slice(1))
  .join(' ');
console.log(resultado); // Javascript É Incrível!
```

■ Encadear métodos de string + array é um padrão muito poderoso.

■ string · trim · split · map · join

9 includes, startsWith e endsWith

Fácil · Strings

Dada a string 'Programação em JavaScript é poderosa', verifique: se contém 'JavaScript', se começa com 'Prog' e se termina com 'rosa'. Exiba os três resultados com mensagens descritivas.

```
const f = 'Programação em JavaScript é poderosa';
console.log('Tem JS:', f.includes('JavaScript')); // true
console.log('Começa:', f.startsWith('Prog')); // true
console.log('Termina:', f.endsWith('rosa')); // true
```

■ includes, startsWith e endsWith aceitam um segundo argumento de posição.

■ string · includes · startsWith · endsWith

10 replace, replaceAll e split

Fácil · Strings

Dada a string 'um dois tres quatro cinco', substitua todos os espaços por hífens, converta para maiúsculas e exiba também a versão dividida em array.

```
const s = 'um dois tres quatro cinco';
console.log(s.replaceAll(' ', '-').toUpperCase());
console.log(s.split(' '));
// ['um', 'dois', 'tres', 'quatro', 'cinco']
```

■ replaceAll() substitui todas as ocorrências; replace() só a primeira sem regex /g.

■ string · replaceAll · split · toUpperCase

11 parseInt, parseFloat e Number

Fácil · Números

Converta '42', '3.14', '0xFF' (hexadecimal), '100px' e 'abc' para número usando parseInt, parseFloat e Number. Exiba os resultados e use isNaN() para verificar os inválidos.

```
console.log(parseInt('42')); // 42
console.log(parseFloat('3.14')); // 3.14
console.log(parseInt('0xFF', 16)); // 255
console.log(parseInt('100px')); // 100
console.log(isNaN(Number('abc'))); // true
```

■ parseInt aceita uma base (radix) como segundo argumento — ex: base 16 para hex.

■ number · parseInt · parseFloat · isNaN

1
2

Math completo

Fácil · Números

Use o objeto Math para: encontrar o maior entre 5 números, arredondar 4.5 e 4.4, truncar 7.9, calcular raiz quadrada de 144 e o valor absoluto de -42.

```
console.log(Math.max(3,7,1,9,5)); // 9
console.log(Math.round(4.5));      // 5
console.log(Math.round(4.4));      // 4
console.log(Math.trunc(7.9));      // 7
console.log(Math.sqrt(144));       // 12
console.log(Math.abs(-42));        // 42
```

■ *Math.trunc()* remove a parte decimal sem arredondar (diferente de *Math.floor* para negativos).

■ Math · number · round · sqrt · abs

1
3

for loop com acumulador

Fácil · Controle de Fluxo

Use um loop for para calcular a soma de todos os números de 1 a 100 e também o produto de 1 a 10 (fatorial de 10). Exiba ambos os resultados.

```
let soma = 0;
for (let i = 1; i <= 100; i++) soma += i;
console.log(soma); // 5050

let fat = 1;
for (let i = 1; i <= 10; i++) fat *= i;
console.log(fat); // 3628800
```

■ *A soma de 1 a N tem fórmula $N*(N+1)/2$. Use o loop para verificar!*

■ for · acumulador · soma · produto

1
4

while com validação

Fácil · Controle de Fluxo

Use while para simular uma contagem regressiva de 20 até 0, pulando os múltiplos de 3 (use continue). Ao final exiba 'Fim!'.

```
let i = 20;
while (i >= 0) {
  if (i % 3 === 0) { i--; continue; }
  console.log(i);
  i--;
}
console.log('Fim!');
```

■ *continue dentro de while pula para a próxima verificação da condição.*

■ while · continue · módulo

1
5

Criar e manipular array

Fácil · Arrays

Crie um array com 5 frutas. Adicione uma fruta no início com unshift, uma no final com push, remova a do início com shift e a do final com pop. Exiba o array após cada operação.

```
const f = ['maçã', 'banana', 'laranja', 'uva', 'manga'];  
f.unshift('abacaxi'); console.log(...f);  
f.push('kiwi'); console.log(...f);  
f.shift(); console.log(...f);  
f.pop(); console.log(...f);
```

■ *unshift/push adicionam; shift/pop removem. Os quatro modificam o array original.*

■ array · push · pop · shift · unshift

1
6

forEach com índice e lógica

Fácil · Arrays

Dado um array de preços, use forEach para exibir cada preço com seu índice, indicando '(mais caro)' no maior valor e '(mais barato)' no menor valor.

```
const p = [29.9, 49.9, 9.9, 99.9, 15.0];  
const max = Math.max(...p), min = Math.min(...p);  
p.forEach((v, i) => {  
  const tag = v===max?' (mais caro)':v===min?' (mais barato)':'';  
  console.log(`${i+1}. R$ ${v.toFixed(2)}${tag}`);  
});
```

■ *Math.max(...arr) e Math.min(...arr) com spread encontram extremos de um array.*

■ array · forEach · Math · spread

1
7

Criar e acessar objeto

Fácil · Objetos

Crie um objeto carro com marca, modelo, ano e cor. Acesse propriedades com notação ponto e colchetes, adicione uma nova propriedade 'km' e delete a propriedade 'cor'.

```
const carro = { marca:'Toyota', modelo:'Corolla', ano:2022, cor:'prata' };  
console.log(carro.marca);  
console.log(carro['modelo']);  
carro.km = 15000;  
delete carro.cor;  
console.log(carro);
```

■ *delete remove uma propriedade; o operador in verifica se ela existe: 'km' in carro.*

■ objeto · propriedades · delete · in

1
8

Object.keys, values e entries

Fácil · Objetos

Dado um objeto produto com 5 propriedades, use `Object.keys()`, `Object.values()` e `Object.entries()` para listar chaves, valores e pares. Calcule também o número de propriedades.

```
const p = { nome: 'Notebook', preco: 3500, estoque: 12, marca: 'Dell', peso: 1.8 };
console.log(Object.keys(p));
console.log(Object.values(p));
Object.entries(p).forEach(([k, v]) => console.log(`${k}: ${v}`));
console.log('Props:', Object.keys(p).length);
```

■ *Object.entries() é ideal para iterar sobre objetos sem for...in.*

■ objeto · Object.keys · Object.values · Object.entries

1
9

Função com múltiplos retornos

Fácil · Funções

Crie uma função `classificarNumero(n)` que retorna 'positivo', 'negativo' ou 'zero' conforme o valor. Crie também a versão arrow function numa linha.

```
function classificarNumero(n) {
  if (n > 0) return 'positivo';
  if (n < 0) return 'negativo';
  return 'zero';
}
const classif = n => n > 0 ? 'positivo' : n < 0 ? 'negativo' : 'zero';
console.log(classif(-5)); // negativo
```

■ *Early return evita else desnecessário — cada condição 'sai' logo que satisfeita.*

■ function · return · ternário

2
0

Arrow function e parâmetro padrão

Fácil · Funções

Crie uma arrow function `calcularDesconto(preco, desconto = 0.1)` que retorna o preço final após aplicar o percentual de desconto. Teste com e sem o segundo argumento.

```
const calcularDesconto = (preco, desc = 0.1) =>
  +(preco * (1 - desc)).toFixed(2);
console.log(calcularDesconto(100)); // 90
console.log(calcularDesconto(200, 0.25)); // 150
```

■ *O + antes de toFixed converte a string de volta para número.*

■ arrow function · parâmetro padrão · toFixed

2
1

for...of com string e array

Fácil · Controle de Fluxo

Use for...of para iterar sobre uma string e contar quantas letras maiúsculas ela contém. Em seguida, itere sobre um array de objetos e exiba apenas os que têm uma propriedade 'ativo' igual a true.

```
let maiusculas = 0;
for (const c of 'Hello World JS') {
  if (c >= 'A' && c <= 'Z') maiusculas++;
}
console.log(maiusculas); // 3

const itens = [{n:'A',ativo:true},{n:'B',ativo:false}];
for (const {n, ativo} of itens)
  if (ativo) console.log(n);
```

■ *for...of funciona com qualquer iterável: strings, arrays, Maps, Sets.*

■ for...of · string · array · destructuring

2
2

slice e substring

Fácil · Strings

Dada a string 'Fundamentos de JavaScript', extraia a primeira palavra, a última palavra e a palavra do meio usando slice(). Exiba cada resultado.

```
const s = 'Fundamentos de JavaScript';
const palavras = s.split(' ');
console.log(palavras[0]); // Fundamentos
console.log(palavras[palavras.length-1]); // JavaScript
console.log(palavras[Math.floor(palavras.length/2)]); // de
```

■ *Combine split com índices para navegar pelas palavras de uma frase.*

■ string · slice · split · array

2
3

indexOf e lastIndexOf

Fácil · Strings

Dada a string 'banana', encontre a primeira e a última ocorrência da letra 'a'. Verifique também se a string contém 'nan' e qual a posição.

```
const s = 'banana';
console.log(s.indexOf('a')); // 1
console.log(s.lastIndexOf('a')); // 5
console.log(s.indexOf('nan')); // 2
console.log(s.includes('nan')); // true
```

■ *indexOf retorna -1 se não encontrar. Use isso em condicionais: if (s.indexOf('x') !== -1).*

■ string · indexOf · lastIndexOf · includes

2
4

padStart, padEnd e repeat

Fácil · Strings

Formate um número de protocolo como '0042', um código com sufixo como 'ID001---' e crie uma linha separadora com '-' repetido 30 vezes.

```
const n = 42;
console.log(String(n).padStart(4, '0')); // 0042
console.log('ID001'.padEnd(8, '-')); // ID001---
console.log('-'.repeat(30));
```

■ *padStart e padEnd são ótimos para formatar tabelas e exibições alinhadas.*

■ string · padStart · padEnd · repeat

2
5

Tipos e typeof

Fácil · Fundamentos

Crie uma função identificarTipo(valor) que retorna uma string descrevendo o tipo real do valor, tratando corretamente null (que typeof retorna 'object') e arrays.

```
const identificarTipo = v => {
  if (v === null) return 'null';
  if (Array.isArray(v)) return 'array';
  return typeof v;
};
console.log(identificarTipo(null)); // null
console.log(identificarTipo([])); // array
console.log(identificarTipo(42)); // number
```

■ *Array.isArray() é a forma correta de detectar arrays — typeof [] retorna 'object'.*

■ typeof · null · Array.isArray · tipos

2
6

Operadores lógicos com curto-circuito

Fácil · Fundamentos

Demonstre curto-circuito: use || para fornecer valor padrão a uma variável possivelmente vazia, e && para executar código apenas se uma condição for verdadeira. Mostre os dois padrões.

```
const nome = '' || 'Anônimo';
console.log(nome); // Anônimo

const logado = true;
logado && console.log('Bem-vindo!'); // executa

const user = null;
const email = user && user.email; // não lança erro
console.log(email); // null
```

■ *|| retorna o primeiro truthy; && retorna o primeiro falsy ou o último valor.*

■ operadores lógicos · curto-circuito · truthy · falsy

2
7

Array com find, findIndex e includes

Fácil · Arrays

Dado um array de objetos de estudantes com nome e nota, encontre o primeiro com nota acima de 8, seu índice, e verifique se algum tem nota exatamente 10.

```
const alunos = [  
  {nome: 'Ana', nota: 7},  
  {nome: 'Bob', nota: 9},  
  {nome: 'Cia', nota: 10},  
];  
console.log(alunos.find(a => a.nota > 8));      // Bob  
console.log(alunos.findIndex(a => a.nota > 8)); // 1  
console.log(alunos.some(a => a.nota === 10));   // true
```

■ *find retorna o elemento, findIndex retorna o índice, some verifica se algum passa no teste.*

■ array · find · findIndex · some

2
8

Destructuring de objeto com renomeação

Fácil · Objetos

Dado um objeto API com campos snake_case (primeiro_nome, ultimo_nome, data_nascimento), use destructuring renomeando as variáveis para camelCase local.

```
const api = { primeiro_nome: 'João', ultimo_nome: 'Silva', data_nascimento: '1990-05-20' };  
const {  
  primeiro_nome: primeiroNome,  
  ultimo_nome: ultimoNome,  
  data_nascimento: dataNascimento  
} = api;  
console.log(primeiroNome, ultimoNome);
```

■ *Na destructuring: { chave: novoNome } extrai e renomeia ao mesmo tempo.*

■ objeto · destructuring · renomeação

2
9

Destructuring de array com skip

Fácil · Arrays

Dado um array [vermelho, verde, azul, amarelo, roxo], extraia apenas a primeira, terceira e quinta cores usando destructuring, ignorando as demais posições.

```
const [vermelho, , azul, , roxo] = ['vermelho', 'verde', 'azul', 'amarelo', 'roxo'];  
console.log(vermelho); // vermelho  
console.log(azul);     // azul  
console.log(roxo);     // roxo
```

■ *Use vírgulas sem variável para pular posições no destructuring de array.*

■ array · destructuring · skip

3
0

Rest parameters e spread

Fácil · Funções

Crie uma função `primeiroEResto(primeiro, ...resto)` que exibe o primeiro elemento separado dos demais. Em seguida, use `spread` para combinar dois arrays sem duplicatas.

```
const primeiroEResto = (primeiro, ...resto) => {  
  console.log('Primeiro:', primeiro);  
  console.log('Resto:', resto);  
};  
primeiroEResto(1, 2, 3, 4); // Primeiro: 1, Resto: [2,3,4]  
  
const a = [1,2,3], b = [3,4,5];  
const unido = [...new Set([...a,...b])];  
console.log(unido); // [1,2,3,4,5]
```

■ *...rest no parâmetro coleta; ...spread na expressão expande.*

■ rest · spread · Set · array

3
1

toFixed, toPrecision e formatação

Fácil · Números

Formate o número 1234567.891 como: 2 casas decimais (`toFixed`), 6 dígitos significativos (`toPrecision`), e como moeda brasileira usando `Intl.NumberFormat`.

```
const n = 1234567.891;  
console.log(n.toFixed(2));           // 1234567.89  
console.log(n.toPrecision(6));       // 1234570  
const fmt = new Intl.NumberFormat('pt-BR', {style: 'currency', currency: 'BRL'});  
console.log(fmt.format(n));          // R$ 1.234.567,89
```

■ *Intl.NumberFormat é a forma moderna e internacionalizada de formatar números.*

■ number · toFixed · toPrecision · Intl

3
2

do...while com acumulador

Fácil · Controle de Fluxo

Use `do...while` para calcular a soma dos dígitos de um número inteiro positivo (ex: 1234 → 1+2+3+4 = 10). O loop deve processar dígito por dígito.

```
let num = 1234, soma = 0;  
do {  
  soma += num % 10;  
  num = Math.floor(num / 10);  
} while (num > 0);  
console.log(soma); // 10
```

■ *num % 10 extrai o último dígito; Math.floor(num/10) remove-o.*

■ do...while · módulo · Math.floor · número

3
3

Truthy, Falsy e nullish

Fácil · Fundamentos

Crie uma função `obterValor(entrada, padrao)` que retorna `padrao` quando `entrada` for `null` ou `undefined` (usando `??`), mas preserve `0` e `''` como valores válidos. Teste com `null`, `undefined`, `0` e `''`.

```
const obterValor = (entrada, padrao) => entrada ?? padrao;
console.log(obterValor(null, 'N/A'));           // N/A
console.log(obterValor(undefined, 42));         // 42
console.log(obterValor(0, 99));                 // 0 (preserva!)
console.log(obterValor('', 'vazio'));           // '' (preserva!)
```

■ `??` (nullish coalescing) só age em `null/undefined`, ao contrário de `||` que age em qualquer falsy.

■ nullish coalescing · `??` · falsy · truthy

3
4

for...in e verificação de propriedade

Fácil · Objetos

Dado um objeto `configurações`, use `for...in` para listar apenas as propriedades próprias (não herdadas) com `hasOwnProperty()`. Adicione uma propriedade ao `prototype` e verifique a diferença.

```
const cfg = { tema: 'dark', lang: 'pt', fontSize: 14 };
for (const k in cfg) {
  if (cfg.hasOwnProperty(k))
    console.log(`${k} = ${cfg[k]}`);
}
// Boa prática: for...in + hasOwnProperty
```

■ `for...in` itera também propriedades herdadas — sempre filtre com `hasOwnProperty`.

■ `for...in` · `hasOwnProperty` · objeto · `prototype`

3
5

Array.isArray e verificações de tipo

Fácil · Arrays

Crie uma função `processar(entrada)` que aceita tanto uma string quanto um array de strings. Se receber uma string, converta em array com um elemento. Depois, exiba todos os itens com `forEach`.

```
const processar = entrada => {
  const arr = Array.isArray(entrada) ? entrada : [entrada];
  arr.forEach((item, i) => console.log(`${i+1}. ${item}`));
};
processar('único');                       // 1. único
processar(['a', 'b', 'c']);               // 1.a 2.b 3.c
```

■ Aceitar ambos os tipos (string ou array) é um padrão comum em APIs flexíveis.

■ `Array.isArray` · `forEach` · ternário · tipos

3
6

String para array de caracteres únicos

Fácil · Strings

Dada uma string, crie um array com todos os caracteres únicos em ordem de aparecimento, sem repetição, usando spread e Set.

```
const unicos = str => [...new Set([...str])];  
console.log(unicos('banana')); // ['b','a','n']  
console.log(unicos('javascript')); // ['j','a','v','s','c','r','i','p','t']
```

■ Spread numa string cria array de chars; Set remove duplicatas mantendo a ordem de inserção.

■ string · Set · spread · array

3
7

Número para extenso (1-19)

Fácil · Strings

Crie uma função que converte números de 1 a 19 para extenso em português, usando um array como lookup table.

```
const extenso = n => {  
  const tab = ['', 'um', 'dois', 'três', 'quatro', 'cinco', 'seis',  
    'sete', 'oito', 'nove', 'dez', 'onze', 'doze', 'treze', 'quatorze',  
    'quinze', 'dezesesseis', 'dezesete', 'dezoito', 'dezenove'];  
  return tab[n] || 'fora do intervalo';  
};  
console.log(extenso(7)); // sete  
console.log(extenso(15)); // quinze
```

■ Arrays como lookup table (mapeamento por índice) são mais simples que switch com 20 cases.

■ array · índice · lookup · string

3
8

Verificar anagrama

Fácil · Strings

Crie uma função que verifica se duas palavras são anagramas (mesmas letras, ordem diferente), ignorando maiúsculas e espaços.

```
const ehAnagrama = (a, b) => {  
  const ord = s => [...s.toLowerCase().replace(/\s/g, '')].sort().join('');  
  return ord(a) === ord(b);  
};  
console.log(ehAnagrama('Roma', 'Amor')); // true  
console.log(ehAnagrama('listen', 'silent')); // true  
console.log(ehAnagrama('hello', 'world')); // false
```

■ Ordenar os caracteres de ambas as strings e comparar é a solução clássica.

■ string · sort · spread · array

3
9

Converter array em objeto

Fácil · Objetos

Dado um array de pares [chave, valor], converta-o em objeto usando `Object.fromEntries()`. Depois, inverta o processo com `Object.entries()`.

```
const pares = [['nome', 'Ana'], ['idade', 25], ['cidade', 'Porto']];
const obj = Object.fromEntries(pares);
console.log(obj); // { nome: 'Ana', idade: 25, cidade: 'Porto' }
console.log(Object.entries(obj));
// de volta ao array de pares
```

■ *`Object.fromEntries` e `Object.entries` são operações inversas — muito úteis para transformar dados.*

■ objeto · `Object.fromEntries` · `Object.entries` · array

4
0

Verificar se objeto está vazio

Fácil · Objetos

Crie uma função `estaVazio(obj)` que retorna `true` se o objeto não tem propriedades próprias. Teste com objeto vazio, com propriedades e com `null`.

```
const estaVazio = obj =>
  obj !== null && typeof obj === 'object' &&
  Object.keys(obj).length === 0;
console.log(estaVazio({})); // true
console.log(estaVazio({a:1})); // false
console.log(estaVazio(null)); // false
```

■ *Sempre valide se é realmente um objeto antes de chamar `Object.keys`.*

■ objeto · `Object.keys` · validação · `typeof`

4
1

Trocar valores sem variável temporária

Fácil · Fundamentos

Troque os valores de duas variáveis usando destructuring de array. Depois implemente uma função `swap(arr, i, j)` que troca dois elementos de um array no lugar.

```
let a = 5, b = 10;
[a, b] = [b, a];
console.log(a, b); // 10 5

const swap = (arr, i, j) => {
  [arr[i], arr[j]] = [arr[j], arr[i]];
};
const v = [1, 2, 3];
swap(v, 0, 2);
console.log(v); // [3, 2, 1]
```

■ *Destructuring para swap é mais limpo e elegante do que usar variável temporária.*

■ destructuring · swap · array · variável

4
2

Contar palavras

Fácil · Strings

Crie uma função `contarPalavras(str)` que conta o número de palavras numa string, ignorando espaços extras no início, fim e entre palavras.

```
const contarPalavras = str =>
  str.trim().split(/\s+/).filter(Boolean).length;
console.log(contarPalavras(' olá mundo js ')); // 3
console.log(contarPalavras('')); // 0
console.log(contarPalavras('uma')); // 1
```

■ `\s+` em regex captura um ou mais espaços em branco (espaço, tab, newline).

■ `string` · `split` · `regex` · `trim` · `filter`

4
3

Gerar sequência com `Array.from`

Fácil · Arrays

Use `Array.from` para gerar: um array de quadrados de 1 a 10, um array de letras de 'a' a 'e' e um array de booleanos alternados (`true`, `false`, `true`...).

```
const quadrados = Array.from({length:10}, (_,i) => (i+1)**2);
console.log(quadrados);

const letras = Array.from({length:5}, (_,i) => String.fromCharCode(97+i));
console.log(letras); // ['a','b','c','d','e']

const bools = Array.from({length:6}, (_,i) => i%2===0);
console.log(bools);
```

■ `String.fromCharCode(97)` retorna 'a'; 97 é o código ASCII de 'a'.

■ `Array.from` · `String.fromCharCode` · `spread` · `gerador`

4
4

Optional chaining e nullish

Fácil · Objetos

Dado um array de objetos usuário onde alguns podem não ter endereço ou telefone, use optional chaining e `??` para exibir informações de forma segura.

```
const users = [
  { nome: 'Ana', endereco: { cidade: 'Porto' } },
  { nome: 'Bob' },
  { nome: 'Cia', telefone: null },
];
users.forEach(u => {
  const cidade = u?.endereco?.cidade ?? 'N/A';
  const tel = u?.telefone ?? 'Sem telefone';
  console.log(`${u.nome}: ${cidade}, ${tel}`);
});
```

■ `?.` e `??` juntos são a combinação perfeita para acessar dados que podem ser nulos.

■ `optional chaining` · `nullish coalescing` · `objeto` · `array`

4
5

Formatação com Intl

Fácil · Números

Formate a data 2024-01-15 nos estilos: data completa em português, data curta, e hora. Formate também o número 9876543.21 como moeda em EUR.

```
const d = new Date(2024, 0, 15);
console.log(d.toLocaleDateString('pt-PT', {dateStyle: 'full'}));
console.log(d.toLocaleDateString('pt-PT', {dateStyle: 'short'}));

const num = 9876543.21;
console.log(new Intl.NumberFormat('pt-PT',
  {style: 'currency', currency: 'EUR'}).format(num));
```

■ Intl é a API nativa do JS para internacionalização — não precisa de bibliotecas externas.

■ Intl · Date · formatação · número · string

4
6

Inverter string

Fácil · Strings

Crie uma função `inverterString(str)` que retorna a string invertida. Resolva de duas formas: usando `spread+reverse+join` e usando um loop `for`.

```
const inverterString = str => [...str].reverse().join('');
console.log(inverterString('JavaScript')); // tpircSavaJ

const inverterLoop = str => {
  let r = '';
  for (let i = str.length-1; i >= 0; i--) r += str[i];
  return r;
};
console.log(inverterLoop('hello')); // olleh
```

■ Strings em JS são imutáveis — você não pode fazer `str.reverse()` diretamente.

■ string · spread · reverse · join · loop

4
7

Flatten com flat

Fácil · Arrays

Dado um array de arrays de notas de turmas, use `flat(1)` para obter todas as notas num array único, depois calcule a média.

```
const turmas = [[7,8,9],[6,7,10],[5,8,9,7]];
const notas = turmas.flat();
console.log(notas);

const media = notas.reduce((s,n) => s+n, 0) / notas.length;
console.log(media.toFixed(2));
```

■ `flat(1)` achata um nível; `flat(Infinity)` achata todos os níveis.

■ array · flat · reduce · média

4
8

Spread em objetos

Fácil · Objetos

Use spread para: clonar um objeto pessoa, mesclar dois objetos de configuração (resolvendo conflito de chaves) e criar uma cópia atualizada com um campo modificado sem mutar o original.

```
const p = {nome: 'Ana', idade: 25};
const clone = {...p};
const cfg = {...{tema: 'dark', lang: 'pt'}, ...{lang: 'en', fonte: 14}};
console.log(cfg); // lang: 'en' vence
const atualizado = {...p, idade: 26};
console.log(p.idade); // 25 (original intacto)
console.log(atualizado.idade); // 26
```

■ Quando chaves se repetem no spread, a última declarada prevalece.

■ objeto · spread · clone · merge · imutabilidade

4
9

Soma de array com reduce vs forEach

Fácil · Arrays

Calcule a soma, o maior e o menor valor de um array de números usando tanto forEach quanto reduce. Compare as abordagens.

```
const nums = [3, 1, 7, 2, 9, 4, 6];
let soma = 0, max = -Infinity, min = Infinity;
nums.forEach(n => { soma += n; if (n > max) max = n; if (n < min) min = n; });
console.log(soma, max, min); // 32 9 1

const soma2 = nums.reduce((a, n) => a + n, 0);
console.log(soma2); // 32
```

■ reduce é mais declarativo; forEach com variáveis externas é mais imperativo — ambos são válidos.

■ array · reduce · forEach · acumulador

5
0

every, some e none

Fácil · Arrays

Dado um array de temperaturas diárias, verifique: se todas são positivas (every), se alguma passa de 35 graus (some), e implemente uma função none() que retorna true se nenhum elemento passar no teste.

```
const temps = [18, 22, 35, 28, 15, 31];
console.log(temps.every(t => t > 0)); // true
console.log(temps.some(t => t > 35)); // false
```

```
const none = (arr, fn) => !arr.some(fn);
console.log(none(temps, t => t < 0)); // true (nenhum negativo)
```

■ none não existe nativamente, mas é facilmente implementado como negação de some.

■ array · every · some · none · funcional

■ Exercícios Médio

5
1

map — transformar objetos

Médio · Arrays

Dado um array de produtos com preço em dólar, use `map()` para criar um novo array com os preços convertidos para euro (1 USD = 0.92 EUR), arredondados para 2 casas, e com a propriedade 'moeda' adicionada.

```
const produtos = [{nome: 'A', preco: 10}, {nome: 'B', preco: 25.5}];
const emEuros = produtos.map(p => ({
  ...p,
  preco: +(p.preco * 0.92).toFixed(2),
  moeda: 'EUR'
}));
console.log(emEuros);
```

■ *`map()` com spread de objeto cria novos objetos sem mutar o original.*

■ array · map · spread · objeto

5
2

filter com múltiplas condições

Médio · Arrays

Dado um array de produtos com nome, preço e categoria, filtre apenas os que são da categoria 'eletronicos', têm preço abaixo de 500 e nome com mais de 5 caracteres.

```
const prods = [
  {nome: 'Tablet', preco: 399, cat: 'eletronicos'},
  {nome: 'TV', preco: 999, cat: 'eletronicos'},
  {nome: 'Fone', preco: 149, cat: 'acessorios'},
  {nome: 'Teclado', preco: 249, cat: 'eletronicos'},
];
const r = prods.filter(p =>
  p.cat==='eletronicos' && p.preco<500 && p.nome.length>5);
console.log(r);
```

■ *Encadeie condições no filter com && — o elemento só passa se todas forem verdadeiras.*

■ array · filter · múltiplas condições

5
3

reduce — estatísticas

Médio · Arrays

Dado um array de notas, use um único reduce para calcular simultaneamente: soma, média, maior nota e menor nota. Retorne um objeto com todas as estatísticas.

```
const notas = [7, 9, 5, 8, 6, 10, 4];
const stats = notas.reduce((acc, n, i, arr) => ({
  soma: acc.soma + n,
  media: (acc.soma + n) / arr.length,
  max: Math.max(acc.max, n),
  min: Math.min(acc.min, n),
}), {soma:0, media:0, max:-Infinity, min:Infinity});
console.log(stats);
```

■ Um único reduce pode calcular múltiplas estatísticas ao mesmo tempo — mais eficiente que vários loops.

■ array · reduce · objeto · estatísticas

5
4

Encadeamento map+filter+reduce

Médio · Arrays

Dado um array de pedidos com itens e quantidades, filtre os pedidos acima de R\$100, some os preços de cada item, e calcule o total geral em uma cadeia.

```
const pedidos = [
  {id:1, itens:[{p:30,q:2},{p:20,q:1}]},
  {id:2, itens:[{p:10,q:3}]},
  {id:3, itens:[{p:60,q:2}]},
];
const total = pedidos
  .map(o => o.itens.reduce((s,i) => s+i.p*i.q, 0))
  .filter(t => t > 100)
  .reduce((s,t) => s+t, 0);
console.log(total); // 150
```

■ Encadear map+filter+reduce é um pipeline funcional de processamento de dados.

■ array · map · filter · reduce · pipeline

5
5

sort com objetos e localeCompare

Médio · Arrays

Dado um array de livros com título, autor e ano, ordene primeiro por ano decrescente, depois por título alfabético em caso de empate. Use localeCompare para suportar acentuação.

```
const livros = [
  {titulo:'Clean Code', autor:'Martin', ano:2008},
  {titulo:'Atomic Habits', autor:'Clear', ano:2018},
  {titulo:'Refactoring', autor:'Fowler', ano:2018},
];
livros.sort((a,b) => b.ano-a.ano || a.titulo.localeCompare(b.titulo,'pt'));
livros.forEach(l => console.log(l.titulo, l.ano));
```

■ localeCompare com locale 'pt' trata corretamente acentos em português.

■ array · sort · localeCompare · objeto · múltiplos critérios

5
6

flat e flatMap avançados

Médio · Arrays

Dado um array de usuários onde cada um tem um array de habilidades, use flatMap para obter todas as habilidades únicas de todos os usuários.

```
const devs = [  
  {nome: 'Ana', skills: ['JS', 'CSS', 'React']},  
  {nome: 'Bob', skills: ['JS', 'Node', 'SQL']},  
  {nome: 'Cia', skills: ['React', 'TypeScript']},  
];  
const unicas = [...new Set(devs.flatMap(d => d.skills))];  
console.log(unicas);
```

■ flatMap extrai e achata arrays aninhados; Set remove as duplicatas resultantes.

■ array · flatMap · Set · spread · objeto

5
7

Closure — contador avançado

Médio · Funções

Use closure para criar uma fábrica de contadores que suporta: incrementar por passo customizável, resetar ao valor inicial, e retornar o histórico de valores como array.

```
const criarContador = (inicial=0, passo=1) => {  
  let v = inicial;  
  const hist = [inicial];  
  return {  
    inc: () => { v += passo; hist.push(v); return v; },  
    reset: () => { v = inicial; hist.push(v); return v; },  
    historico: () => [...hist],  
  };  
};  
const c = criarContador(0, 5);  
c.inc(); c.inc(); c.reset();  
console.log(c.historico()); // [0,5,10,0]
```

■ O closure mantém 'v' e 'hist' privados — acessíveis apenas via métodos retornados.

■ closure · estado privado · fábrica · array

Crie uma IIFE que simula um módulo de configuração: lê variáveis de ambiente (simuladas), fornece valores padrão com ??, e expõe métodos get() e set() com validação.

```
const Config = (() => {  
  const env = { PORT:'3000', NODE_ENV:undefined };  
  const cfg = {  
    port: +(env.PORT ?? 8080),  
    env: env.NODE_ENV ?? 'development',  
  };  
  return {  
    get: k => cfg[k],  
    set: (k,v) => { if(k in cfg) cfg[k]=v; },  
  };  
})();  
console.log(Config.get('port')); // 3000
```

■ IIFE com retorno de objeto expõe uma API pública mantendo o estado interno privado.

■ IIFE · closure · módulo · ?? · validação

Crie uma função estatisticas(...numeros) que usa spread para receber qualquer quantidade de argumentos e retorna um objeto com min, max, soma, média e mediana.

```
const estatisticas = (...nums) => {  
  const sorted = [...nums].sort((a,b)=>a-b);  
  const n = nums.length;  
  const mid = Math.floor(n/2);  
  const mediana = n%2 ? sorted[mid] : (sorted[mid-1]+sorted[mid])/2;  
  return {  
    min: sorted[0], max: sorted[n-1],  
    soma: nums.reduce((s,v)=>s+v,0),  
    media: nums.reduce((s,v)=>s+v,0)/n,  
    mediana,  
  };  
};  
console.log(estatisticas(3,1,7,2,5));
```

■ A mediana é o valor do meio; se o array tiver tamanho par, é a média dos dois centrais.

■ rest · spread · sort · reduce · mediana

Use loops aninhados com label para buscar o primeiro par (i,j) onde $i*j > 50$ em uma matriz 10x10, saindo de ambos os loops ao encontrar.

```
let achou = null;
outer:
for (let i = 1; i <= 10; i++) {
  for (let j = 1; j <= 10; j++) {
    if (i * j > 50) {
      achou = {i, j, prod: i*j};
      break outer;
    }
  }
}
console.log(achou); // {i:6, j:9, prod:54}
```

■ *Labels em loops permitem break/continue saírem de loops externos diretamente.*

■ for · break · label · loop aninhado

Dado um CSV de uma linha ('João,25,Lisboa,Engenheiro'), converta em objeto com as chaves ['nome','idade','cidade','profissao']. Depois converta o objeto de volta para CSV.

```
const csv = 'João,25,Lisboa,Engenheiro';
const keys = ['nome', 'idade', 'cidade', 'profissao'];
const obj = Object.fromEntries(
  csv.split(',').map((v,i) => [keys[i], v])
);
console.log(obj);
const back = Object.values(obj).join(',');
console.log(back);
```

■ *Combinar split + map + Object.fromEntries é uma forma elegante de parsear CSV simples.*

■ string · split · join · Object.fromEntries · CSV

6
2

Contar frequência com Map

Médio · Objetos

Dada uma frase, conte a frequência de cada palavra usando um Map (não um objeto simples). Exiba as 3 palavras mais frequentes.

```
const frase = 'o gato e o rato e o cão';
const freq = new Map();
frase.split(' ').forEach(p => freq.set(p, (freq.get(p)||0)+1));
const top3 = [...freq.entries()]
  .sort((a,b) => b[1]-a[1])
  .slice(0,3);
console.log(top3);
```

■ *Map é superior ao objeto simples como dicionário: aceita qualquer tipo de chave.*

■ Map · frequência · sort · string · slice

6
3

Adicionar métodos e this

Médio · Objetos

Crie um objeto contaBancaria com saldo, titular e métodos depositar(), sacar() (com validação) e extrato(). Use this para referenciar o próprio objeto.

```
const conta = {
  titular: 'Ana Silva',
  saldo: 1000,
  depositar(v) { this.saldo += v; return this; },
  sacar(v) {
    if (v > this.saldo) { console.log('Saldo insuficiente'); return this; }
    this.saldo -= v;
    return this;
  },
  extrato() { console.log(`${this.titular}: R${this.saldo}`); return this; }
};
conta.depositar(500).sacar(200).extrato();
```

■ *Retornar this nos métodos permite method chaining — padrão fluent interface.*

■ objeto · this · method chaining · validação

6
4

Computed properties e Object.assign

Médio · Objetos

Crie uma função `atualizarCampo(obj, campo, valor)` que retorna um novo objeto com o campo atualizado usando `computed properties`. Depois use `Object.assign` para mesclar múltiplas fontes.

```
const atualizarCampo = (obj, campo, valor) =>
  ({ ...obj, [campo]: valor });

const base = {a:1, b:2};
const novo = atualizarCampo(base, 'b', 99);
console.log(novo); // {a:1, b:99}

const merged = Object.assign({}, base, {c:3}, {d:4});
console.log(merged);
```

■ *[campo] como chave cria uma propriedade dinamicamente — essencial em code dinâmico.*

■ objeto · `computed properties` · `Object.assign` · `spread` · imutável

6
5

Object.entries e transformação

Médio · Objetos

Dado um objeto de preços em dólar, use `Object.entries()` e `map()` para criar um novo objeto com todos os preços convertidos para euro (x 0.92) e arredondados.

```
const precos = { teclado: 49.99, mouse: 29.99, monitor: 399.99 };
const emEuros = Object.fromEntries(
  Object.entries(precos).map(([k,v]) => [k, +(v*0.92).toFixed(2)])
);
console.log(emEuros);
```

■ *Object.entries + map + Object.fromEntries é o 'map de objetos' em JavaScript.*

■ objeto · `Object.entries` · `Object.fromEntries` · `map`

6
6

Função que retorna função e closure

Médio · Funções

Crie uma função `criarSaudacao(idioma)` que retorna uma função de saudação específica para aquele idioma. Suporte pelo menos inglês, português e espanhol.

```
const criarSaudacao = idioma => {
  const msgs = { pt:'Olá', en:'Hello', es:'Hola' };
  const saud = msgs[idioma] || 'Hi';
  return nome => `${saud}, ${nome}!`;
};

const ola = criarSaudacao('pt');
const hello = criarSaudacao('en');
console.log(ola('Ana')); // Olá, Ana!
console.log(hello('Bob')); // Hello, Bob!
```

■ *A função interna acessa 'saud' do escopo externo via closure — mesmo após retornar.*

■ closure · higher-order function · objeto · idioma

6
7

Verificar primo e listar primos

Médio · Números

Crie uma função `ehPrimo(n)` e use-a com `filter()` para gerar todos os primos até 100. Exiba também quantos há no intervalo.

```
const ehPrimo = n => {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++)
    if (n % i === 0) return false;
  return true;
};
const primos = Array.from({length:100}, (_, i) => i+1).filter(ehPrimo);
console.log(primos.length); // 25
console.log(primos);
```

■ *Passar o nome da função diretamente para `filter` sem wrapper é mais limpo: `.filter(ehPrimo)`.*

■ `number` · `Math` · `filter` · `Array.from` · algoritmo

6
8

Merge e diff de arrays com Set

Médio · Arrays

Dado dois arrays de tags, calcule: união (todos os únicos), interseção (presentes em ambos) e diferença simétrica (presentes em apenas um).

```
const a = ['js', 'css', 'html', 'react'];
const b = ['js', 'python', 'css', 'node'];
const sa = new Set(a), sb = new Set(b);
const uniao = [...new Set([...a, ...b])];
const inter = a.filter(x => sb.has(x));
const difSim = [...a.filter(x => !sb.has(x)), ...b.filter(x => !sa.has(x))];
console.log('União:', uniao);
console.log('Interseção:', inter);
console.log('Dif. simétrica:', difSim);
```

■ *`Set.has()` é $O(1)$ — muito mais eficiente que `Array.includes()` para buscas.*

■ `Set` · `array` · união · interseção · diferença · teoria dos conjuntos

Implemente Fibonacci com memoização usando um objeto como cache. Compare o tempo de execução com e sem memoização para `fib(40)`.

```
const cache = {};  
const fib = n => {  
  if (n in cache) return cache[n];  
  if (n <= 1) return (cache[n] = n);  
  return (cache[n] = fib(n-1) + fib(n-2));  
};  
console.time('memoizado');  
console.log(fib(40));  
console.timeEnd('memoizado');  
// Sem memo: exponencial  $O(2^n)$ . Com memo: linear  $O(n)$ .
```

■ *console.time/timeEnd é uma forma simples de medir performance sem bibliotecas.*

■ memoização · recursão · performance · objeto · cache

Dado um array de departamentos com subdepartamentos aninhados em múltiplos níveis, achate a estrutura para um único array de nomes de departamentos.

```
const deps = ['TI', ['Dev', ['Front', 'Back'], 'QA'], ['RH', 'Financeiro']];  
console.log(deps.flat(Infinity));  
// ['TI', 'Dev', 'Front', 'Back', 'QA', 'RH', 'Financeiro']
```

// Alternativa recursiva manual:

```
const achatar = arr =>  
  arr.reduce((a,v) => a.concat(Array.isArray(v) ? achatar(v) : v), []);  
console.log(achatar(deps));
```

■ *flat(Infinity) é a solução moderna; a alternativa recursiva com reduce é didática.*

■ array · flat · recursão · reduce

7
1

Número aleatório e simulação

Médio · Números

Crie funções para: gerar inteiro aleatório num intervalo, simular lançamento de dado N vezes e calcular a frequência de cada face.

```
const rand = (min,max) => Math.floor(Math.random()*(max-min+1))+min;
const lancamentos = Array.from({length:600}, () => rand(1,6));
const freq = lancamentos.reduce((acc,v) => {
  acc[v] = (acc[v]||0)+1;
  return acc;
}, {});
Object.entries(freq).forEach(([f,c]) =>
  console.log(`Face ${f}: ${c} vezes (${(c/600*100).toFixed(1)}%`)`
);
```

■ Com 600 lançamentos, cada face deveria aparecer ~16,7% das vezes (Lei dos Grandes Números).

■ Math.random · reduce · frequência · simulação

7
2

splice e reorganização de array

Médio · Arrays

Dado um array de tarefas ordenadas por prioridade, use splice para: mover a última tarefa para a primeira posição, inserir 2 novas tarefas no meio e remover tarefas duplicadas.

```
const t = ['baixa', 'média', 'alta', 'urgente'];
const ultima = t.splice(t.length-1,1)[0];
t.unshift(ultima);
console.log(t); // ['urgente', 'baixa', 'média', 'alta']
t.splice(2,0,'crítica', 'bloqueante');
console.log(t);
```

■ splice retorna os elementos removidos como array — guarde-os se precisar usá-los.

■ array · splice · unshift · reorganização

7
3

Remove duplicatas com Set e Map

Médio · Arrays

Dado um array de objetos com id e nome (com duplicatas por id), remova as duplicadas mantendo apenas a primeira ocorrência, usando Map como dicionário.

```
const dados = [
  {id:1, nome:'Ana'}, {id:2, nome:'Bob'},
  {id:1, nome:'Ana2'}, {id:3, nome:'Cia'}
];
const mapa = new Map();
dados.forEach(d => { if (!mapa.has(d.id)) mapa.set(d.id, d); });
const unicos = [...mapa.values()];
console.log(unicos);
```

■ Map garante ordem de inserção e desempenho O(1) para has/set/get.

■ Map · Set · array · duplicatas · objeto

7
4

Palíndromo e manipulação de string

Médio · Strings

Crie uma função que verifica palíndromo ignorando espaços, acentos e pontuação. Teste com 'Arara', 'A man a plan a canal Panama' e 'JavaScript'.

```
const ehPalindromo = str => {
  const limpa = str.normalize('NFD')
    .replace(/[\u0300-\u036f\s\W]/g, '').toLowerCase();
  return limpa === [...limpa].reverse().join('');
};
console.log(ehPalindromo('Arara')); // true
console.log(ehPalindromo('A man a plan a canal Panama')); // true
console.log(ehPalindromo('JavaScript')); // false
```

■ *normalize('NFD') + regex de diacríticos remove acentos antes de comparar.*

■ string · palíndromo · regex · normalize · reverse

7
5

Capitalizar, camelCase e kebab-case

Médio · Strings

Crie funções para converter uma string para: Title Case, camelCase e kebab-case. Teste com 'hello world from javascript'.

```
const str = 'hello world from javascript';
const titleCase = s => s.split(' ').map(p=>p[0].toUpperCase()+p.slice(1)).join(' ');
const camelCase = s => s.split(' ').map((p,i)=>i?p[0].toUpperCase()+p.slice(1):p).join('');
const kebabCase = s => s.split(' ').join('-');
console.log(titleCase(str)); // Hello World From Javascript
console.log(camelCase(str)); // helloWorldFromJavascript
console.log(kebabCase(str)); // hello-world-from-javascript
```

■ *Essas conversões são comuns em código que precisa transformar identificadores.*

■ string · split · map · join · camelCase · kebab

7
6

Agrupar e contar por propriedade

Médio · Arrays

Dado um array de transações com tipo ('crédito'/'débito') e valor, agrupe por tipo e calcule o total de cada grupo e o saldo final.

```
const trans = [
  {tipo:'crédito',v:500},{tipo:'débito',v:150},
  {tipo:'crédito',v:300},{tipo:'débito',v:80},
];
const grupos = trans.reduce((acc,t) => {
  acc[t.tipo] = (acc[t.tipo]||0) + t.v;
  return acc;
}, {});
console.log(grupos);
console.log('Saldo:', grupos.crédito - grupos.débito);
```

■ *reduce como agrupador é um padrão fundamental para análise de dados em JS.*

■ array · reduce · agrupamento · objeto · saldo

Crie um Map de capitais europeias (país → capital). Use for...of para listar os pares, encontrar países que começam com 'P' e converter para um array de objetos.

```
const capitais = new Map([
  ['Portugal', 'Lisboa'], ['França', 'Paris'], ['Polônia', 'Varsóvia']
]);
for (const [pais, cap] of capitais)
  console.log(`${pais}: ${cap}`);

const comP = [...capitais]
  .filter(([p]) => p.startsWith('P'))
  .map(([pais, capital]) => ({pais, capital}));
console.log(comP);
```

■ Map é iterável com for...of retornando [chave, valor] — ideal para destructuring.

■ Map · for...of · destructuring · filter · spread

Dado um objeto de configuração profundamente aninhado com muitos campos opcionais, crie uma função getConfig(obj) que extrai todos os valores com defaults seguros usando ?? e ?..

```
const getConfig = obj => ({
  host: obj?.server?.host ?? 'localhost',
  port: obj?.server?.port ?? 3000,
  debug: obj?.logging?.debug ?? false,
  maxConn: obj?.db?.maxConnections ?? 10,
});
console.log(getConfig({}));
console.log(getConfig({server:{port:8080}}));
```

■ Encadear ?. com ?? cria um sistema robusto de defaults sem if/else aninhados.

■ optional chaining · nullish coalescing · objeto · defaults

Use `Array.from` para converter: um `Set` em array ordenado, um `Map` em array de chaves, e a propriedade `arguments` de uma função legada em array moderno.

```
const s = new Set([3,1,4,1,5,9,2]);
console.log(Array.from(s).sort((a,b)=>a-b));

const m = new Map([['a',1],['b',2]]);
console.log(Array.from(m.keys()));

function legada() {
  const args = Array.from(arguments);
  return args.reduce((s,v)=>s+v,0);
}
console.log(legada(1,2,3,4)); // 10
```

■ *Array.from converte qualquer iterável — Set, Map, NodeList, arguments, strings.*

■ `Array.from` · `Set` · `Map` · `arguments` · iterável

Dado um sistema de pedidos, use `every` para verificar se todos os itens têm estoque disponível, `some` para verificar se algum tem frete grátis, e implemente uma validação completa.

```
const pedido = {
  itens: [
    {nome:'A', estoque:5, freteGratis:false},
    {nome:'B', estoque:2, freteGratis:true},
    {nome:'C', estoque:0, freteGratis:false},
  ]
};

const temEstoque = pedido.itens.every(i => i.estoque > 0);
const freteGratis = pedido.itens.some(i => i.freteGratis);
console.log('Pode finalizar:', temEstoque);
console.log('Frete grátis:', freteGratis);
```

■ *every para validação global, some para verificação de ao menos um caso especial.*

■ `array` · `every` · `some` · `objeto` · `validação`

Crie uma função `formatarTabela(dados)` que exibe um array de objetos como tabela alinhada no console, usando `padEnd` para colunas fixas.

```
const dados = [
  {nome: 'Ana', cargo: 'Dev', salario: 5000},
  {nome: 'Bruno', cargo: 'Designer', salario: 4200},
  {nome: 'Carlos', cargo: 'PM', salario: 6000},
];

dados.forEach(({nome, cargo, salario}) => {
  console.log(
    nome.padEnd(10) +
    cargo.padEnd(12) +
    `R${salario.toFixed(2)}`.padStart(10)
  );
});
```

■ `padEnd` alinha à esquerda (texto); `padStart` alinha à direita (números).

■ string · padEnd · padStart · formatação · tabela

Use regex para: validar um email simples, extrair todos os números de uma string mista, e verificar se uma senha tem pelo menos 8 chars, 1 número e 1 maiúscula.

```
const validarEmail = e => /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/ .test(e);
console.log(validarEmail('user@email.com')); // true

const extrairNums = s => s.match(/\\d+/g) || [];
console.log(extrairNums('a1b22c333')); // ['1', '22', '333']

const validarSenha = s => /^(?=.*\\d)(?=.*[A-Z]).{8,}$/ .test(s);
console.log(validarSenha('Senha123')); // true
```

■ Lookahead `(?=.*\\d)` verifica se existe um dígito em qualquer posição sem consumir chars.

■ string · regex · match · test · validação

Use Symbol para criar propriedades verdadeiramente únicas em um objeto, demonstrando que dois Symbols com a mesma descrição são diferentes, e que não aparecem em `Object.keys()`.

```
const ID = Symbol('id');
const ID2 = Symbol('id');
console.log(ID === ID2); // false (únicos!)

const user = { nome: 'Ana', [ID]: 42 };
console.log(Object.keys(user)); // ['nome'] – ID não aparece
console.log(user[ID]); // 42
console.log(Object.getOwnPropertySymbols(user)); // [Symbol(id)]
```

■ Symbols são invisíveis ao `for...in`, `JSON.stringify` e `Object.keys` — ideais para metadados.

■ Symbol · objeto · propriedades · chave única

Demonstre o uso de `WeakMap` para associar metadados privados a objetos sem impedir o garbage collection, e `WeakSet` para rastrear objetos 'visitados'.

```
const metadados = new WeakMap();
const obj1 = { nome: 'Ana' };
metadados.set(obj1, { criado: Date.now(), versao: 1 });
console.log(metadados.get(obj1));
// obj1 = null; // metadados seria coletado pelo GC

const visitados = new WeakSet();
const pagina = { url: '/home' };
visitados.add(pagina);
console.log(visitados.has(pagina)); // true
```

■ `WeakMap/WeakSet` não impedem garbage collection — perfeitos para cache de objetos de vida curta.

■ WeakMap · WeakSet · objeto · garbage collection · metadados

Crie um objeto `range` que é iterável (implementa `Symbol.iterator`), permitindo usar `for...of` para iterar num intervalo `[inicio, fim]` com passo customizável.

```
const range = (inicio, fim, passo=1) => ({
  [Symbol.iterator]() {
    let atual = inicio;
    return {
      next() {
        if (atual <= fim) {
          const v = atual; atual += passo;
          return { value:v, done:false };
        }
        return { value:undefined, done:true };
      }
    };
  }
});

for (const n of range(0,10,2)) console.log(n); // 0,2,4,6,8,10
```

■ *`Symbol.iterator` torna qualquer objeto iterável com `for...of`, `spread` e `destructuring`.*

■ `Symbol.iterator` · iterador · protocolo · `for...of`

Use `Proxy` para criar um objeto com validação automática: o setter deve lançar erro se `'idade'` for negativa ou `'nome'` for vazio.

```
const handler = {
  set(alvo, prop, valor) {
    if (prop === 'idade' && valor < 0) throw new RangeError('Idade inválida');
    if (prop === 'nome' && !valor.trim()) throw new TypeError('Nome vazio');
    alvo[prop] = valor;
    return true;
  }
};

const user = new Proxy({}, handler);
user.nome = 'Ana';
user.idade = 25;
console.log(user);
try { user.idade = -1; } catch(e) { console.log(e.message); }
```

■ *`Proxy` pode interceptar `get`, `set`, `has`, `deleteProperty` e outros — base para reatividade (Vue 3).*

■ `Proxy` · validação · objeto · `trap` · setter

Crie um generator function que produz a sequência de Fibonacci infinita. Use next() para extrair os primeiros 10 valores.

```
function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}

const gen = fibonacci();
const primeiros10 = Array.from({length:10}, () => gen.next().value);
console.log(primeiros10);
// [0,1,1,2,3,5,8,13,21,34]
```

■ *Generators são funções pausáveis — yield suspende e next() retoma a execução.*

■ generator · function* · yield · Fibonacci · iterador

Crie uma classe de erro customizada IdadeInvalidaError. Crie uma função validarIdade(n) que lança esse erro. Use try/catch/finally para tratar e exibir o resultado.

```
class IdadeInvalidaError extends Error {
  constructor(msg) {
    super(msg);
    this.name = 'IdadeInvalidaError';
  }
}

const validarIdade = n => {
  if (typeof n !== 'number') throw new IdadeInvalidaError('Deve ser número');
  if (n < 0 || n > 150) throw new IdadeInvalidaError(`Fora do intervalo: ${n}`);
  return n;
};

try {
  validarIdade(-5);
} catch(e) {
  console.log(e.name, e.message);
} finally {
  console.log('Validação concluída');
}
```

■ *Erros customizados (extends Error) permitem instanceof checks e mensagens específicas.*

■ try · catch · finally · Error · classe · extends

Crie uma classe `Animal` com nome e som. Estenda com classes `Cachorro` e `Gato` que sobrescrevem o método `falar()`. Use `super()` e `instanceof`.

```
class Animal {
  constructor(nome) { this.nome = nome; }
  falar() { return `${this.nome} faz um som.`; }
}
class Cachorro extends Animal {
  falar() { return `${this.nome} late: Au au!`; }
}
class Gato extends Animal {
  falar() { return `${this.nome} mia: Miau!`; }
}
const pets = [new Cachorro('Rex'), new Gato('Mia')];
pets.forEach(p => console.log(p.falar()));
console.log(pets[0] instanceof Animal); // true
```

■ *`super()` chama o construtor da classe pai; necessário antes de usar `this` em classes filhas.*

■ class · extends · super · herança · instanceof · polimorfismo

Crie uma classe `Temperatura` com um valor interno em Celsius. Adicione getters/setters para Fahrenheit e Kelvin que convertem automaticamente.

```
class Temperatura {
  constructor(celsius) { this._c = celsius; }
  get celsius() { return this._c; }
  set celsius(v) { this._c = v; }
  get fahrenheit() { return this._c * 9/5 + 32; }
  set fahrenheit(v) { this._c = (v - 32) * 5/9; }
  get kelvin() { return this._c + 273.15; }
}
const t = new Temperatura(0);
console.log(t.fahrenheit); // 32
console.log(t.kelvin);     // 273.15
t.fahrenheit = 212;
console.log(t.celsius);    // 100
```

■ *Getters e setters permitem propriedades calculadas com sintaxe de acesso simples.*

■ class · getter · setter · conversão · encapsulamento

Crie uma classe Matematica com métodos estáticos para: fatorial, combinação (C(n,k)) e verificação de primo. Use os métodos estáticos para calcular C(10,3).

```
class Matematica {
  static fatorial(n) { return n<=1 ? 1 : n * Matematica.fatorial(n-1); }
  static combinacao(n,k) {
    return Matematica.fatorial(n) / (Matematica.fatorial(k)*Matematica.fatorial(n-k));
  }
  static ehPrimo(n) {
    if(n<2) return false;
    for(let i=2;i<=Math.sqrt(n);i++) if(n%i===0) return false;
    return true;
  }
}
console.log(Matematica.combinacao(10,3)); // 120
console.log(Matematica.ehPrimo(17));      // true
```

■ Métodos static pertencem à classe, não às instâncias — não precisam de new.

■ class · static · fatorial · recursão · combinação

Crie uma função buscarUsuario(id) que retorna uma Promise. Se id > 0, resolve com um objeto; senão rejeita com erro. Encadeie then/catch e adicione um finally.

```
const buscarUsuario = id => new Promise((resolve, reject) => {
  setTimeout(() => {
    if (id > 0)
      resolve({ id, nome: 'Ana', email: 'ana@ex.com' });
    else
      reject(new Error('ID inválido'));
  }, 100);
});

buscarUsuario(1)
  .then(u => console.log('Usuário:', u.nome))
  .catch(e => console.log('Erro:', e.message))
  .finally(() => console.log('Concluído'));
```

■ Promises têm 3 estados: pending → fulfilled (resolve) ou rejected (reject).

■ Promise · then · catch · finally · assíncrono

Reescreva o exercício anterior usando async/await com try/catch. Crie também uma função para buscar múltiplos usuários em paralelo com Promise.all.

```
const buscarUsuario = id => new Promise((res, rej) =>
  setTimeout(() => id>0 ? res({id,nome:`User${id}`}) : rej(new Error('Inválido')), 50)
);

async function main() {
  try {
    const u = await buscarUsuario(1);
    console.log(u.nome);
    const varios = await Promise.all([1,2,3].map(buscarUsuario));
    console.log(varios.map(v=>v.nome));
  } catch(e) {
    console.log('Erro:', e.message);
  }
}
main();
```

■ *Promise.all executa em paralelo e resolve quando TODAS as promessas resolvem.*

■ `async` · `await` · `try` · `catch` · `Promise.all` · paralelo

Use Promise.allSettled para processar um array de Promises onde algumas falham, sem que uma falha cancele as outras. Compare com Promise.race.

```
const p = ms => new Promise((res, rej) =>
  setTimeout(() => ms>0 ? res(`ok-${ms}`) : rej('erro'), Math.abs(ms))
);

Promise.allSettled([p(100),p(-50),p(200)]).then(results => {
  results.forEach(r =>
    console.log(r.status, r.value||r.reason)
  );
});

Promise.race([p(300),p(100),p(200)]).then(v => console.log('Mais rápido:', v));
```

■ *allSettled nunca rejeita — ótimo para batch com possíveis falhas. race retorna a primeira.*

■ `Promise.allSettled` · `Promise.race` · `async` · paralelo

Use regex com grupos nomeados para extrair componentes de uma data no formato 'DD/MM/AAAA' e reformatá-la para ISO 'AAAA-MM-DD'.

```
const reformatar = str => {
  const m = str.match(/(?<d>\d{2})\/(?<m>\d{2})\/(?<a>\d{4})/);
  if (!m) return null;
  const {d, m:mes, a} = m.groups;
  return `${a}-${mes}-${d}`;
};
console.log(reformatar('25/12/2024')); // 2024-12-25
console.log(reformatar('01/01/2000')); // 2000-01-01
```

■ Grupos nomeados (?...) são acessados via `match.groups.nome` — código muito legível.

■ regex · grupos nomeados · match · string · data

Use `Intl.Collator` para ordenar nomes com acentos corretamente, e `Intl.PluralRules` para exibir '1 item' vs '5 itens'.

```
const nomes = ['Álvaro', 'Bruno', 'Ângela', 'Carlos', 'Ágata'];
const ord = new Intl.Collator('pt').compare;
console.log(nomes.sort(ord));
// ['Ágata', 'Álvaro', 'Ângela', 'Bruno', 'Carlos']

const plural = new Intl.PluralRules('pt-BR');
const itens = n => plural.select(n) === 'one' ? `${n} item` : `${n} itens`;
console.log(itens(1)); // 1 item
console.log(itens(5)); // 5 itens
```

■ *Intl.Collator* é a forma correta de ordenar strings com acentos em qualquer idioma.

■ Intl · Collator · PluralRules · sort · string · acentos

Implemente o padrão Observer: um Subject que mantém uma lista de observers e os notifica com dados quando seu estado muda.

```
const criarSubject = () => {
  const obs = [];
  return {
    subscribe: fn => obs.push(fn),
    unsubscribe: fn => obs.splice(obs.indexOf(fn), 1),
    notify: data => obs.forEach(fn => fn(data)),
  };
};

const preco = criarSubject();
const log = d => console.log('Log:', d);
const alert = d => console.log('Alerta! Novo preço:', d);
preco.subscribe(log);
preco.subscribe(alert);
preco.notify(150.00);
preco.unsubscribe(alert);
preco.notify(120.00);
```

■ *Observer desacopla quem produz dados de quem os consome — base de sistemas reativos.*

■ Observer · padrão de design · closure · array · funcional

Demonstre a diferença entre funções puras e impuras. Refatore 3 funções impuras (com efeitos colaterais) em versões puras. Explique o impacto nos testes.

```
// Impura: modifica estado externo
let total = 0;
const addImpura = v => { total += v; };

// Pura: sem efeitos colaterais
const addPura = (total, v) => total + v;

// Impura: depende de estado externo
const agora = () => Date.now(); // impura (resultado varia)

// Pura: mesma entrada = mesma saída
const dobrar = x => x * 2; // pura
console.log(dobrar(5)); // sempre 10
console.log(addPura(0, 5)); // sempre 5
```

■ *Funções puras são previsíveis e testáveis — pilar da programação funcional.*

■ função pura · efeitos colaterais · funcional · imutabilidade

Implemente `pipe()` e `compose()` (da direita para a esquerda). Use-os para criar pipelines de processamento de texto: `trim` → `lowercase` → `split` → `filter` → `join`.

```
const pipe = (...fns) => x => fns.reduce((v, f) => f(v), x);
const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x);

const processar = pipe(
  s => s.trim(),
  s => s.toLowerCase(),
  s => s.split(' '),
  arr => arr.filter(p => p.length > 2),
  arr => arr.join('-')
);
console.log(processar(' Olá Mundo de JavaScript '));
// olá-mundo-javascript
```

■ *pipe executa esquerda→direita (como `|>` no shell); compose executa direita→esquerda (matemática).*

■ pipe · compose · reduce · funcional · composição

Implemente uma função `memoize` genérica que usa `Map` como cache e suporta `TTL` (time-to-live): entradas expiram após `N` milissegundos.

```
const memoize = (fn, ttl=Infinity) => {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    const hit = cache.get(key);
    if (hit && Date.now()-hit.ts < ttl) return hit.val;
    const val = fn(...args);
    cache.set(key, { val, ts: Date.now() });
    return val;
  };
};

const lento = memoize(n => { /* simula cálculo */ return n*n; }, 5000);
console.log(lento(5)); // calcula
console.log(lento(5)); // do cache
```

■ *Cache com TTL é essencial para dados que mudam com o tempo (ex: cotações, sessões).*

■ memoize · Map · cache · TTL · performance · closure

■ Exercícios Difícil

1

0 Closure — contador privado avançado

Difícil · Funções

1

Use closure para criar um contador com estado privado e múltiplas instâncias independentes, expondo incrementar(), decrementar(), resetar() e valor().

```
const criarContador = (inicial=0) => {
  let count = inicial;
  return {
    incrementar: () => ++count,
    decrementar: () => --count,
    resetar: () => (count = inicial),
    valor: () => count,
  };
};
const c1 = criarContador(10);
const c2 = criarContador(100);
c1.incrementar(); c1.incrementar();
c2.decrementar();
console.log(c1.valor(), c2.valor()); // 12 99
```

■ Cada chamada de criarContador() cria um closure independente — estados isolados.

■ closure · estado privado · fábrica · objeto

1

0 Debounce com suporte a cancel e flush

Difícil · Funções

2

Implemente debounce(fn, delay) com métodos cancel() (cancela pendente) e flush() (executa imediatamente se pendente).

```
const debounce = (fn, delay) => {
  let timer, lastArgs;
  const exec = (...args) => {
    clearTimeout(timer);
    lastArgs = args;
    timer = setTimeout(() => { fn(...args); timer=null; }, delay);
  };
  exec.cancel = () => { clearTimeout(timer); timer=null; };
  exec.flush = () => { if(timer) { clearTimeout(timer); fn(...lastArgs); timer=null; } };
  return exec;
};
const buscar = debounce(q => console.log('Buscando:', q), 300);
buscar('a'); buscar.flush(); // executa imediatamente
```

■ cancel() e flush() são recursos do debounce do Lodash — úteis em testes e cleanup.

■ closure · debounce · cancel · flush · higher-order

1
0
3

Pipe — composição funcional

Difícil · Funções

Implemente `pipe()` e use-o para criar um processador de dados: converter string CSV em array de objetos, filtrar, transformar e formatar a saída.

```
const pipe = (...fns) => x => fns.reduce((v, f) => f(v), x);

const processar = pipe(
  csv => csv.trim().split('\n'),
  linhas => linhas.map(l => l.split(','))
    .map(([n, i]) => ({nome:n.trim(), idade:+i})),
  dados => dados.filter(d => d.idade >= 18),
  dados => dados.sort((a, b) => a.nome.localeCompare(b.nome))
);

const csv = `Ana,22\nBob,16\nCarlos,30`;
console.log(processar(csv));
```

■ *`pipe()` transforma uma série de transformações em um único fluxo legível.*

■ pipe · composição · CSV · funcional · reduce

1
0
4

Tabela de multiplicação como matriz

Difícil · Arrays

Gere uma matriz 10x10 da tabuada com `Array.from`. Implemente também uma função que exibe a matriz formatada como tabela no console.

```
const tabela = Array.from({length:10}, (_, i) =>
  Array.from({length:10}, (_, j) => (i+1)*(j+1))
);

console.log(tabela[2][3]); // 12

tabela.forEach((row, i) => {
  const linha = row.map(v => String(v).padStart(4)).join(' ');
  console.log(`${i+1}:${linha}`);
});
```

■ *`padStart(4)` alinha números de forma que a tabela fique visualmente organizada.*

■ array · Array.from · matriz · padStart · formatação

1
0
5

Zip e unzip

Difícil · Arrays

Implemente `zip(a, b)` que combina dois arrays em pares e `unzip(arr)` que separa os pares em dois arrays. Suporte N arrays com `zipN(...arrays)`.

```
const zip = (a,b) =>
  Array.from({length:Math.min(a.length,b.length)}, (_,i)=>[a[i],b[i]]);
const unzip = pares => [
  pares.map(([a])=>a), pares.map([,b])=>b
];
const zipN = (...arrs) => {
  const len = Math.min(...arrs.map(a=>a.length));
  return Array.from({length:len}, (_,i) => arrs.map(a=>a[i]));
};
console.log(zip([1,2,3],['a','b','c']));
console.log(zipN([1,2],[3,4],[5,6]));
```

■ *zipN generaliza zip para N arrays usando map em cada índice.*

■ array · zip · unzip · Array.from · funcional

1
0
6

Deep clone sem JSON

Difícil · Objetos

Implemente `deepClone(obj)` sem `JSON.parse/stringify`, preservando `Date`, `RegExp`, `Map`, `Set` e arrays aninhados.

```
const deepClone = obj => {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Date) return new Date(obj);
  if (obj instanceof RegExp) return new RegExp(obj);
  if (obj instanceof Map) return new Map([...obj].map(([k,v])=>[deepClone(k),deepClone(v)]));
  ;
  if (obj instanceof Set) return new Set([...obj].map(deepClone));
  if (Array.isArray(obj)) return obj.map(deepClone);
  return Object.fromEntries(Object.entries(obj).map(([k,v])=>[k,deepClone(v)]));
};
const orig = { d:new Date(), arr:[1,[2,3]], m:new Map([['a',1]]) };
const c = deepClone(orig);
console.log(c.d !== orig.d); // true – objetos diferentes
```

■ *JSON.parse/stringify destrói Date, undefined, functions, RegExp, Map e Set.*

■ objeto · recursão · Date · Map · Set · clone profundo

1
0
7

Run-Length Encoding (RLE)

Difícil · Strings

Implemente compressão e descompressão RLE. 'aaabbbcc' ↔ 'a3b3c2'. A descompressão deve reconstruir a string original.

```
const comprimir = str =>
  str.replace(/(.)\1*/g, (m,c,rep) => c+(rep.length+1>1?rep.length+1:'));

const descomprimir = str =>
  str.replace(/(\D)(\d*)/g, (_,c,n) => c.repeat(n?+n:1));

console.log(comprimir('aaabbbcc')); // a3b3c2
console.log(comprimir('abcd'));    // abcd
console.log(descomprimir('a3b3c2')); // aaabbbcc
```

■ A descompressão é tão importante quanto a compressão — sempre implemente os dois lados.

■ string · regex · replace · RLE · compressão

1
0
8

Currying genérico

Difícil · Funções

Implemente `curry(fn)` genérico que suporta chamada parcial com qualquer número de argumentos por vez. Teste com funções de 2, 3 e 4 parâmetros.

```
const curry = fn => {
  const ar = fn.length;
  return function curried(...args) {
    if (args.length >= ar) return fn(...args);
    return (...mais) => curried(...args, ...mais);
  };
};

const soma4 = curry((a,b,c,d) => a+b+c+d);
console.log(soma4(1)(2)(3)(4)); // 10
console.log(soma4(1,2)(3)(4));  // 10
console.log(soma4(1)(2,3)(4));  // 10
```

■ *fn.length* retorna a aridade (parâmetros declarados). Permite currying automático.

■ curry · closure · aridade · higher-order · spread

1
0
9

Chunk com reduce

Difícil · Arrays

Implemente `chunk(arr, size)` de duas formas: com `Array.from+slice` e com `reduce`. Compare as abordagens e teste com array não divisível pelo tamanho.

```
const chunkFrom = (arr, size) =>
  Array.from({length:Math.ceil(arr.length/size)},
    (_,i) => arr.slice(i*size, i*size+size));

const chunkReduce = (arr, size) =>
  arr.reduce((acc, v, i) => {
    if (i%size===0) acc.push([]);
    acc[acc.length-1].push(v);
    return acc;
  }, []);

console.log(chunkFrom([1,2,3,4,5,6,7], 3));
// [[1,2,3],[4,5,6],[7]]
console.log(chunkReduce([1,2,3,4,5], 2));
```

■ *Array.from+slice é mais declarativo; reduce é mais flexível para adaptações.*

■ array · chunk · Array.from · reduce · slice

1
1
0

Diff de objetos profundo

Difícil · Objetos

Implemente `diff(obj1, obj2)` que retorna as diferenças entre dois objetos, incluindo objetos aninhados, indicando campos adicionados, removidos e modificados.

```
const diff = (a, b) => {
  const chaves = new Set([...Object.keys(a), ...Object.keys(b)]);
  return Object.fromEntries(
    [...chaves].reduce((acc, k) => {
      if (!(k in a)) acc.push([k, {op:'add',valor:b[k]}]);
      else if (!(k in b)) acc.push([k, {op:'del',valor:a[k]}]);
      else if (JSON.stringify(a[k])!==JSON.stringify(b[k]))
        acc.push([k, {op:'mod', de:a[k], para:b[k]}]);
      return acc;
    }, [])
  );
};

console.log(diff({x:1,y:2,z:3},{x:1,y:9,w:4}));
```

■ *Set de todas as chaves garante que detectamos adições e remoções em ambos os lados.*

■ objeto · diff · Set · reduce · Object.fromEntries

1
1
1

Once e limitador de chamadas

Difícil · Funções

Implemente `once(fn)` que executa apenas uma vez, e `limite(fn, n)` que executa no máximo N vezes, retornando `undefined` nas chamadas excedentes.

```
const once = fn => {
  let chamado=false, resultado;
  return (...args) => {
    if (!chamado) { chamado=true; resultado=fn(...args); }
    return resultado;
  };
};

const limite = (fn, n) => {
  let count = 0;
  return (...args) => count++ < n ? fn(...args) : undefined;
};

const inicializar = once(() => 'iniciado!');
console.log(inicializar(), inicializar()); // iniciado! iniciado!
const max2 = limite(x=>x*2, 2);
console.log(max2(5), max2(6), max2(7)); // 10 12 undefined
```

■ *once e limite são utilitários de controle de execução usados em Lodash/Underscore.*

■ closure · once · limite · higher-order · estado

1
1
2

Interseção e diferença simétrica

Difícil · Arrays

Implemente interseção, diferença, diferença simétrica e produto cartesiano para arrays. Use Set internamente para eficiência $O(1)$.

```
const inter = (a,b) => { const sb=new Set(b); return a.filter(x=>sb.has(x)); };
const dif = (a,b) => { const sb=new Set(b); return a.filter(x=>!sb.has(x)); };
const difSim = (a,b) => [...dif(a,b), ...dif(b,a)];
const cart = (a,b) => a.flatMap(x=>b.map(y=>[x,y]));

const A=[1,2,3,4,5], B=[3,4,5,6,7];
console.log(inter(A,B)); // [3,4,5]
console.log(dif(A,B)); // [1,2]
console.log(difSim(A,B)); // [1,2,6,7]
console.log(cart([1,2],['a','b']));
```

■ *Set.has() é $O(1)$ vs Array.includes() $O(n)$ — para arrays grandes, a diferença é enorme.*

■ Set · array · interseção · diferença · cartesiano · $O(1)$

1
1
3

Flatten de objeto aninhado

Difícil · Objetos

Implemente `flatten(obj)` que transforma objetos aninhados em planos (notação ponto) e `unflatten(obj)` que reconstrói o objeto aninhado.

```
const flatten = (obj, pref='') =>
  Object.entries(obj).reduce((acc, [k,v]) => {
    const key = pref ? `${pref}.${k}` : k;
    return typeof v==='object' && v && !Array.isArray(v)
      ? {...acc, ...flatten(v, key)}
      : {...acc, [key]: v};
  }, {});

const unflatten = obj =>
  Object.entries(obj).reduce((acc, [k,v]) => {
    k.split('.').reduce((o,p,i,arr) => {
      return o[p] ??= i===arr.length-1 ? v : {};
    }, acc);
    return acc;
  }, {});

const obj = {a:{b:{c:1},d:2},e:3};
const plano = flatten(obj);
console.log(plano);
console.log(unflatten(plano));
```

■ *unflatten usa reduce com ??= para criar a estrutura aninhada progressivamente.*

■ objeto · recursão · flatten · unflatten · reduce

Crie a sequência Fibonacci de 3 formas: iterativa, recursiva memoizada e como generator infinito. Compare performance para fib(40).

```
// Iterativa O(n)
const fibIter = n => { let [a,b]=[0,1]; for(let i=0;i<n;i++) [a,b]=[b,a+b]; return a; };

// Memoizada
const cache = new Map();
const fibMemo = n => {
  if(n<=1) return n;
  if(cache.has(n)) return cache.get(n);
  const v = fibMemo(n-1)+fibMemo(n-2);
  cache.set(n,v); return v;
};

// Generator
function* fibGen() { let [a,b]=[0,1]; while(true) { yield a; [a,b]=[b,a+b]; } }

console.log(fibIter(40));
console.log(fibMemo(40));
const g = fibGen();
console.log(Array.from({length:41}, ()=>g.next().value)[40]);
```

■ *Iterativa é $O(n)$ espaço $O(1)$; recursiva pura é $O(2^n)$; memoizada é $O(n)$ tempo e espaço.*

■ Fibonacci · iterador · memoize · generator · performance

1
1
5

Ordenação por múltiplos critérios e estável

Difícil · Arrays

Implemente uma função `sortBy(arr, ...critérios)` que ordena um array de objetos por múltiplos campos com direção (`asc/desc`) para cada um.

```
const sortBy = (arr, ...crit) => [...arr].sort((a,b) => {
  for (const [campo, dir='asc'] of crit) {
    const mult = dir==='desc' ? -1 : 1;
    const r = typeof a[campo]==='string'
      ? a[campo].localeCompare(b[campo], 'pt')
      : a[campo]-b[campo];
    if (r!==0) return r*mult;
  }
  return 0;
});

const dados = [{n:'B',i:30},{n:'A',i:25},{n:'C',i:30}];
console.log(sortBy(dados, ['i', 'asc'], ['n', 'asc']).map(d=>d.n));
// ['A', 'B', 'C']
```

■ *sort estável (desde ES2019) mantém a ordem relativa de elementos iguais.*

■ array · sort · localeCompare · múltiplos critérios · estável

1
1
6

Partial application avançado

Difícil · Funções

Implemente `partial(fn, ...preArgs)` e `partialRight(fn, ...sufixArgs)` que pré-definem argumentos do início e do fim respectivamente. Combine com `curry`.

```
const partial = (fn, ...pre) => (...rest) => fn(...pre, ...rest);
const partialRight = (fn, ...suf) => (...rest) => fn(...rest, ...suf);

const dividir = (a,b) => a/b;
const dividirPor2 = partialRight(dividir, 2);
const dividir10Por = partial(dividir, 10);

console.log(dividirPor2(10)); // 5 (10/2)
console.log(dividir10Por(5)); // 2 (10/5)

const log = partial(console.log, '[INFO]');
log('Sistema iniciado'); // [INFO] Sistema iniciado
```

■ *partialRight é útil quando você quer fixar o divisor/base, não o dividendo.*

■ partial · partialRight · closure · spread · funcional

1
1
7

Produto cartesiano N-dimensional

Difícil · Arrays

Implemente `cartesiano(...arrays)` que funciona com qualquer número de arrays, gerando todas as combinações possíveis.

```
const cartesiano = (...arrs) =>
  arrs.reduce((acc, arr) =>
    acc.flatMap(combo => arr.map(item => [...combo, item])),
    [[]]
  );

console.log(cartesiano(['P', 'M'], ['azul', 'preto'], ['algodao']));
// [['P', 'azul', 'algodao'], ['P', 'preto', 'algodao'],
//   ['M', 'azul', 'algodao'], ['M', 'preto', 'algodao']]
```

■ Começar com `[[] ]` (array com combo vazio) é o truque para `reduce` acumular combinações.

■ array · cartesiano · flatMap · reduce · combinatória

1
1
8

Immutable update profundo

Difícil · Objetos

Implemente `setIn(obj, caminho, valor)` que atualiza uma propriedade profunda imutavelmente, dado um caminho como array de chaves.

```
const setIn = (obj, [head, ...tail], valor) =>
  tail.length === 0
    ? {...obj, [head]: valor}
    : {...obj, [head]: setIn(obj[head] ?? {}, tail, valor)};

const estado = {user:{perfil:{nome:'Ana',pontos:100}}};
const novo = setIn(estado, ['user','perfil','pontos'], 150);
console.log(estado.user.perfil.pontos); // 100 (imutável)
console.log(novo.user.perfil.pontos);    // 150
```

■ `setIn` é o coração de bibliotecas imutáveis como `Immer` e o que `Redux` faz internamente.

■ objeto · spread · imutabilidade · recursão · caminho

1
1
9

Parser de query string robusto

Difícil · Strings

Implemente `parseQS(str)` e `stringifyQS(obj)` para converter entre query strings e objetos, com suporte a arrays (chave[]) e valores aninhados.

```
const parseQS = str =>
  str.replace(/^\?/, '').split('&')
    .reduce((acc, par) => {
      const [k,v=''] = par.split('=').map(decodeURIComponent);
      if (k.endsWith('[]')) {
        const ch = k.slice(0,-2);
        (acc[ch] ??= []).push(v);
      } else acc[k] = v;
    }, {});
const stringifyQS = obj =>
  Object.entries(obj).flatMap(([k,v]) =>
    Array.isArray(v) ? v.map(i=>`${k}[]={encodeURIComponent(i)}`)
      : [`${k}=${encodeURIComponent(v)}`]
  ).join('&');
console.log(parseQS('nome=Ana&tags[]=js&tags[]=css'));
```

■ Na prática use `URLSearchParams`, mas implementar do zero é excelente exercício de parsing.

■ string · split · reduce · encodeURIComponent · query string

Implemente `throttle(fn, ms, {leading, trailing})` que controla se executa no início e/ou no final do intervalo.

```
const throttle = (fn, ms, {leading=true, trailing=true}={}) => {
  let timer=null, ultimo=0, pendente=null;
  return (...args) => {
    const agora = Date.now();
    if (!ultimo && !leading) ultimo = agora;
    const restante = ms - (agora - ultimo);
    if (restante <= 0) {
      if (timer) { clearTimeout(timer); timer=null; }
      ultimo = agora; fn(...args);
    } else if (trailing) {
      clearTimeout(timer);
      timer = setTimeout(() => {
        ultimo = leading ? Date.now() : 0;
        timer = null; fn(...args);
      }, restante);
    }
  };
};

const log = throttle(x=>console.log(x), 1000);
log('a'); log('b');
```

■ *leading=true executa no início; trailing=true executa no fim — Lodash suporta ambos.*

■ [throttle](#) · [closure](#) · [leading](#) · [trailing](#) · [performance](#)

1
2
1

Fisher-Yates shuffle verificado

Difícil · Arrays

Implemente Fisher-Yates shuffle e valide que é imparcial rodando 10000 simulações e verificando que cada posição recebe cada valor com frequência ~igual.

```
const shuffle = arr => {
  const a = [...arr];
  for (let i=a.length-1; i>0; i--) {
    const j = Math.floor(Math.random()*(i+1));
    [a[i],a[j]] = [a[j],a[i]];
  }
  return a;
};

// Validação estatística
const freq = Array.from({length:3}, ()=>Array(3).fill(0));
for (let t=0; t<9000; t++) {
  shuffle([0,1,2]).forEach((v,i) => freq[i][v]++);
}
freq.forEach((row,i) => console.log(`Pos ${i}:`, row.map(c=>(c/9000*100).toFixed(1)+'%')));
```

■ Um shuffle imparcial deve mostrar ~33.3% para cada valor em cada posição.

■ array · shuffle · Fisher-Yates · estatística · Math.random

1
2
2

Pipeline de transformação de dados

Difícil · Objetos

Dado um array de vendas com vendedor, produto, quantidade e preço, calcule: total por vendedor, produto mais vendido, ticket médio, e top 3 vendedores.

```
const vendas = [
  {v: 'Ana', p: 'A', q: 3, preco: 100}, {v: 'Bob', p: 'B', q: 1, preco: 500},
  {v: 'Ana', p: 'B', q: 2, preco: 500}, {v: 'Bob', p: 'A', q: 4, preco: 100},
  {v: 'Cia', p: 'C', q: 5, preco: 200},
];

const totais = vendas.reduce((acc, {v, q, preco}) => {
  acc[v] = (acc[v] || 0) + q*preco; return acc;
}, {});

const top3 = Object.entries(totais).sort((a,b)=>b[1]-a[1]).slice(0,3);
console.log('Totais:', totais);
console.log('Top 3:', top3);
```

■ Encadear reduce + sort + slice é o padrão clássico de ranking em JS.

■ array · reduce · sort · pipeline · objeto · ranking

1
2
3

Tokenizador com estados

Difícil · Strings

Implemente um tokenizador que analisa uma expressão matemática e retorna tokens tipados: NUMBER, OPERATOR, LPAREN, RPAREN.

```
const tokenizar = expr => {
  const tokens = [];
  let i = 0;
  while (i < expr.length) {
    if (/\\s/.test(expr[i])) { i++; continue; }
    const numM = expr.slice(i).match(/^\\d+(\\.\\d+)?/);
    if (numM) { tokens.push({tipo: 'NUMBER', val: +numM[0]}); i+=numM[0].length; continue; }
    if ('+-*/'.includes(expr[i])) { tokens.push({tipo: 'OPERATOR', val: expr[i]}); i++; continue; }
    if (expr[i]=== '(') { tokens.push({tipo: 'LPAREN'}); i++; continue; }
    if (expr[i]=== ')') { tokens.push({tipo: 'RPAREN'}); i++; }
    else i++;
  }
  return tokens;
};
console.log(tokenizar('3.14 + 2 * (10 - 4)'));
```

■ Tokenizadores baseados em estado são o primeiro passo de qualquer parser/compilador.

■ string · regex · tokenizador · parser · while

1
2
4

Rotação e transformações de array

Difícil · Arrays

Implemente: rotacionar(arr, k) para direita/esquerda, espelhar(matriz) na diagonal principal e transpor(matriz).

```
const rotacionar = (arr, k) => {
  const n=arr.length, p=((k%n)+n)%n;
  return [...arr.slice(n-p), ...arr.slice(0,n-p)];
};
const transpor = m => m[0].map((_, i) => m.map(row=>row[i]));
const espelhar = m => m.map(row=>[...row].reverse());

console.log(rotacionar([1,2,3,4,5], 2)); // [4,5,1,2,3]
const mat = [[1,2,3],[4,5,6],[7,8,9]];
console.log(transpor(mat)); // [[1,4,7],[2,5,8],[3,6,9]]
```

■ Transpor uma matriz: colunas viram linhas. Espelhar: cada linha é invertida.

■ array · rotação · transposição · matriz · slice

1
2
5

Memoize genérico com Map e recursão

Difícil · Funções

Implemente `memoize(fn)` que suporta funções recursivas corretamente, usando `Map` como cache e permitindo que a função memoizada chame a si mesma.

```
const memoize = fn => {
  const cache = new Map();
  const memoizado = (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const r = fn.call(memoizado, ...args);
    cache.set(key, r); return r;
  };
  return memoizado;
};
const fib = memoize(function(n) {
  return n <= 1 ? n : this(n-1) + this(n-2);
});
console.log(fib(50)); // Rápido!
```

■ *`fn.call(memoizado, ...)` faz a função recursiva chamar a versão memoizada de si mesma.*

■ memoize · Map · recursão · closure · performance

1
2
6

Sliding window com deque

Difícil · Arrays

Implemente `maxSliding(arr, k)` eficientemente usando uma fila dupla (deque) para manter $O(n)$ ao invés de $O(n*k)$.

```
const maxSliding = (arr, k) => {
  const result = [], deque = []; // deque armazena índices
  for (let i=0; i<arr.length; i++) {
    while (deque.length && deque[0] <= i-k) deque.shift();
    while (deque.length && arr[deque[deque.length-1]] <= arr[i]) deque.pop();
    deque.push(i);
    if (i >= k-1) result.push(arr[deque[0]]);
  }
  return result;
};
console.log(maxSliding([1,3,2,5,4,6,3], 3));
// [3,5,5,6,6]
```

■ *Deque mantém índices em ordem decrescente de valor — o maior sempre no início.*

■ array · deque · sliding window · $O(n)$ · algoritmo

1
2
7

EventEmitter completo

Difícil · Objetos

Implemente EventEmitter com on(), off(), emit(), once() (ouvinte de única vez) e removeAllListeners().

```
const criarEmitter = () => {
  const listeners = {};
  return {
    on(ev, fn) { (listeners[ev]??=[]).push({fn, once: false}); },
    once(ev, fn) { (listeners[ev]??=[]).push({fn, once: true}); },
    off(ev, fn) { listeners[ev] = (listeners[ev] || []).filter(l => l.fn !== fn); },
    removeAllListeners(ev) { if(ev) delete listeners[ev]; else Object.keys(listeners).forEach(k => delete listeners[k]); },
    emit(ev, ...args) {
      (listeners[ev] || []).forEach(l => l.fn(...args));
      listeners[ev] = (listeners[ev] || []).filter(l => !l.once);
    }
  };
};

const bus = criarEmitter();
bus.once('login', u => console.log('Bem-vindo, ', u));
bus.emit('login', 'Ana'); // executa
bus.emit('login', 'Bob'); // silêncio
```

■ *once remove automaticamente o listener após a primeira execução — essencial em UIs.*

■ EventEmitter · closure · Observer · once · array

1
2
8

Retry com backoff exponencial

Difícil · Funções

Implemente retry(fn, tentativas, delayMs) com backoff exponencial: cada falha dobra o delay. Retorne Promise.

```
const retry = (fn, tentativas=3, delay=100) =>
  new Promise(async (resolve, reject) => {
    for (let i=0; i<tentativas; i++) {
      try { return resolve(await fn()); }
      catch(e) {
        if (i===tentativas-1) return reject(e);
        await new Promise(r=>setTimeout(r, delay*(2**i)));
        console.log(`Tentativa ${i+1} falhou. Aguardando ${delay*(2**i)}ms...`);
      }
    }
  });

let n = 0;
const instavel = () => ++n<3 ? Promise.reject('falha') : Promise.resolve('ok!');
retry(instavel).then(console.log);
```

■ *Backoff exponencial (100ms, 200ms, 400ms...) evita sobrecarregar serviços instáveis.*

■ retry · backoff · Promise · async · await · exponencial

1
2
9

Powerset e combinações

Difícil · Arrays

Implemente: `powerset(arr)` que gera todos os subconjuntos, `combinacoes(arr, k)` que gera combinações de tamanho `k`, e `permutacoes(arr)` que gera todas as permutações.

```
const powerset = arr =>
  arr.reduce((sub,v) => [...sub, ...sub.map(s=>[...s,v])], [[]]);

const combinacoes = (arr, k) =>
  k===0 ? [[]] :
  arr.flatMap((v,i) =>
    combinacoes(arr.slice(i+1),k-1).map(c=>[v,...c]));

const permutacoes = arr =>
  arr.length<=1 ? [arr] :
  arr.flatMap((v,i) =>
    permutacoes([...arr.slice(0,i),...arr.slice(i+1)]).map(p=>[v,...p]));

console.log(powerset([1,2,3]).length);      // 8 = 2^3
console.log(combinacoes([1,2,3,4],2));      // [[1,2],...]
console.log(permutacoes([1,2,3]).length);   // 6 = 3!
```

■ *`powerset` tem 2^n elementos, combinações $C(n,k)$, permutações $n!$ — complexidade cresce rápido.*

■ array · powerset · combinações · permutações · recursão · combinatória

1
3
0

Mini interpretador de expressão

Difícil · Desafio

Implemente um avaliador de expressões matemáticas simples (adição, subtração, multiplicação, divisão) usando o algoritmo shunting-yard para respeitar precedência sem eval.

```
const calcular = expr => {
  if (!/^[\\d\\s+\\-*/.()]+$/i.test(expr)) throw new Error('Inválido');
  return Function('return ' + expr)();
};

// Versão segura com tokens
const prec = {'+':1, '-':1, '*':2, '/':2};
const aplicar = (ops,vals) => {
  const b=vals.pop(),a=vals.pop(),op=ops.pop();
  vals.push(op==='+'?a+b:op==='-'?a-b:op==='*'?a*b:a/b);
};

console.log(calcular('2 + 3 * 4')); // 14
console.log(calcular('(2+3)*4'));  // 20
```

■ *Shunting-yard (Dijkstra) é o algoritmo clássico para parsear expressões com precedência.*

■ string · regex · Function · eval · expressão · parser

1
3
1

Deep freeze e imutabilidade

Difícil · Objetos

Implemente `deepFreeze(obj)` que congela um objeto e todos os seus objetos aninhados. Verifique que nenhuma propriedade pode ser modificada.

```
const deepFreeze = obj => {
  Object.getOwnPropertyNames(obj).forEach(k => {
    if (typeof obj[k] === 'object' && obj[k] !== null)
      deepFreeze(obj[k]);
  });
  return Object.freeze(obj);
};

const cfg = deepFreeze({ db: { host:'localhost', port:5432 }, debug:true });
try {
  cfg.db.port = 9999; // TypeError em strict mode
} catch(e) { console.log('Não pode modificar:', e.message); }
console.log(Object.isFrozen(cfg.db)); // true
```

■ *Object.freeze é raso (shallow) — deepFreeze garante imutabilidade em todos os níveis.*

■ objeto · deepFreeze · Object.freeze · recursão · imutabilidade

1
3
2

Lazy evaluation com thunks

Difícil · Funções

Implemente `lazy(fn)` que adia a execução de `fn` até o primeiro acesso ao valor, cacheando o resultado. Demonstre com cálculo custoso.

```
const lazy = fn => {
  let calculado = false, valor;
  return () => {
    if (!calculado) {
      valor = fn();
      calculado = true;
      console.log('Calculado!');
    }
    return valor;
  };
};

const pesado = lazy(() => {
  // Simula cálculo demorado
  return Array.from({length:1000000}, (_, i)=>i).reduce((a,b)=>a+b, 0);
});

console.log('Antes do acesso');
console.log(pesado()); // Calculado! 499999500000
console.log(pesado()); // (cache, sem recalcular)
```

■ *Thunks são funções de zero argumentos que adiam cálculo — base de sistemas lazy.*

■ lazy · thunk · closure · cache · performance

Implemente transducers: composição de map e filter que processa um array em um único passo, sem criar arrays intermediários.

```
const mapping = fn => reducer => (acc,v) => reducer(acc, fn(v));
const filtering = pred => reducer => (acc,v) => pred(v) ? reducer(acc,v) : acc;
const appender = (acc,v) => { acc.push(v); return acc; };
const transduce = (xform, reducer, init, coll) =>
  coll.reduce(xform(reducer), init);

const xform = filtering(n=>n%2!==0)(mapping(n=>n*10)(appender));
const resultado = transduce(x=>x, appender, [], [1,2,3,4,5]
  .reduce(xform, []));
// Alternativa direta:
const r2 = [1,2,3,4,5].reduce(
  filtering(n=>n%2!==0)(mapping(n=>n*10)(appender)), []);
console.log(r2); // [10,30,50]
```

■ *Transducers compõem transformações sem arrays intermediários — eficientes para grandes datasets.*

■ transducer · funcional · reduce · composição · performance

Implemente um scheduler que executa tarefas em fila com concorrência máxima configurável, usando Promises.

```
const scheduler = (concorrencia=2) => {
  const fila = [], ativos = new Set();
  const executar = async () => {
    while (fila.length && ativos.size < concorrancia) {
      const {fn,resolve,reject} = fila.shift();
      const id = Symbol();
      ativos.add(id);
      fn().then(resolve).catch(reject)
        .finally(() => { ativos.delete(id); executar(); });
    }
  };
  return fn => new Promise((resolve,reject) => {
    fila.push({fn,resolve,reject});
    executar();
  });
};

const sched = scheduler(2);
const tarefa = (n,ms) => () => new Promise(r => setTimeout(()=>{console.log(`T${n}`);r(n);},
ms));
[1,2,3,4].forEach(n => sched(tarefa(n,100*n)));
```

■ *Controlar concorrência com fila e Set de ativos é um padrão de sistemas de tarefas.*

■ Promise · scheduler · concorrência · async · fila · Set

1
3
5

Algoritmo de busca binária

Difícil · Algoritmos

Implemente busca binária iterativa e recursiva. Implemente também buscaInsercao que retorna o índice onde o elemento deveria ser inserido para manter a ordenação.

```
const buscaBin = (arr, alvo) => {
  let [lo,hi] = [0, arr.length-1];
  while (lo<=hi) {
    const mid = (lo+hi)>>1;
    if (arr[mid]===alvo) return mid;
    arr[mid]<alvo ? lo=mid+1 : hi=mid-1;
  }
  return -1;
};

const buscaInsercao = (arr, v) => {
  let [lo,hi] = [0, arr.length];
  while (lo<hi) { const mid=(lo+hi)>>1; arr[mid]<v?lo=mid+1:hi=mid; }
  return lo;
};

console.log(buscaBin([1,3,5,7,9,11],7)); // 3
console.log(buscaInsercao([1,3,5,7,9],6)); // 3
```

■ $(lo+hi)>>1$ é equivalente a $\text{Math.floor}((lo+hi)/2)$ — *bitshift* é mais rápido.

■ busca binária · algoritmo · $O(\log n)$ · recursão · inserção ordenada

1
3
6

LRU Cache

Difícil · Algoritmos

Implemente um LRU Cache (Least Recently Used) com capacidade máxima, usando Map para manter a ordem de acesso.

```
class LRUCache {
  constructor(cap) { this.cap=cap; this.cache=new Map(); }
  get(k) {
    if (!this.cache.has(k)) return -1;
    const v=this.cache.get(k);
    this.cache.delete(k); this.cache.set(k,v);
    return v;
  }
  set(k,v) {
    if (this.cache.has(k)) this.cache.delete(k);
    else if (this.cache.size>=this.cap) this.cache.delete(this.cache.keys().next().value);
    this.cache.set(k,v);
  }
}

const lru = new LRUCache(3);
lru.set('a',1); lru.set('b',2); lru.set('c',3);
lru.get('a');
lru.set('d',4); // remove 'b' (menos recente)
console.log(lru.get('b')); // -1
```

■ *Map preserva a ordem de inserção — mover para o fim ao acessar simula 'mais recente'.*

■ LRU · Map · cache · classe · algoritmo · O(1)

Implemente uma Trie para autocomplete: `insert(palavra)`, `search(palavra)` e `startsWith(prefixo)` que retorna todas as palavras com aquele prefixo.

```
class Trie {
  constructor() { this.root={}; }
  insert(w) {
    let n=this.root;
    for(const c of w) n=n[c]?={};
    n.$=true;
  }
  search(w) {
    let n=this.root;
    for(const c of w) { if(!n[c]) return false; n=n[c]; }
    return !!n.$;
  }
  startsWith(pref) {
    let n=this.root;
    for(const c of pref) { if(!n[c]) return []; n=n[c]; }
    const res=[];
    const dfs=(node,s)=>{ if(node.$) res.push(s); Object.entries(node).filter(([k])=>k!=='$')
    ).forEach(([k,v])=>dfs(v,s+k)); };
    dfs(n,pref); return res;
  }
}

const t=new Trie();
['apple','app','application','apply'].forEach(w=>t.insert(w));
console.log(t.startsWith('app'));
```

■ *Trie oferece busca por prefixo em $O(m)$ onde m é o tamanho do prefixo — ideal para autocomplete.*

■ Trie · árvore · DFS · autocomplete · algoritmo

1
3
8

Expressão regular avançada

Difícil · Strings

Use regex com: lookahead positivo e negativo para validar senha complexa, lookbehind para extrair valores após símbolo de moeda, e replace com função para formatar número com separadores.

```
// Senha: 8+ chars, 1 maiúscula, 1 número, 1 especial
const validarSenha = s =>
  /^(?=.*[A-Z])(?=.*\d)(?=.*[!@#%]).{8,}$/.test(s);
console.log(validarSenha('Senha@123')); // true

// Lookbehind: valor após R$
const extrairValor = s => s.match(/(?<=R$\s?)[\d.,]+/g);
console.log(extrairValor('Total: R$ 1.234,56')); // ['1.234,56']

// Formatar número
const fmtNum = n => n.toString().replace(/\B(?=(\d{3})+(?!\d))/g, '.');
console.log(fmtNum(1234567)); // 1.234.567
```

■ *Lookahead (?!...) e lookbehind (?<=...) são asserções de posição sem consumir caracteres.*

■ regex · lookahead · lookbehind · validação · string · replace

1
3
9

Ordenação topológica

Difícil · Algoritmos

Dado um grafo de dependências (objeto onde cada chave tem array de dependências), implemente ordenação topológica para determinar a ordem de instalação de pacotes.

```
const topoSort = grafo => {
  const visitado=new Set(), resultado=[];
  const dfs = no => {
    if (visitado.has(no)) return;
    visitado.add(no);
    (grafo[no]||[]).forEach(dfs);
    resultado.unshift(no);
  };
  Object.keys(grafo).forEach(dfs);
  return resultado;
};

const deps = {
  app: ['react', 'lodash'],
  react: ['react-dom'],
  'react-dom': [],
  lodash: [],
};

console.log(topoSort(deps));
// ['app', 'react', 'react-dom', 'lodash'] (uma ordem válida)
```

■ *DFS em grafos acíclicos dirigidos produz ordenação topológica — base de bundlers e npm.*

■ grafo · DFS · topológica · Set · recursão · algoritmo

1
4
0

Lazy list (stream infinita)

Difícil · Funções

Implemente uma lista lazy que suporta take(n), map(fn), filter(fn) e pode representar sequências infinitas sem calcular todos os valores.

```
const lazyList = gen => ({
  take: n => Array.from({length:n}, ()=>gen.next().value),
  map: fn => lazyList((function*(){ for(const v of gen) yield fn(v); })()),
  filter: fn => lazyList((function*(){ for(const v of gen) if(fn(v)) yield v; })()),
});
const naturais = (function*() { let n=0; while(true) yield n++; })();
const resultado = lazyList(naturais)
  .filter(n => n%2===0)
  .map(n => n**2)
  .take(5);
console.log(resultado); // [0,4,16,36,64]
```

■ *Generators + lazy wrappers criam streams infinitas eficientes — nenhum valor é calculado antecipadamente.*

■ generator · lazy · stream · filter · map · infinito

1
4
1

Padrão Strategy

Difícil · Objetos

Implemente o padrão Strategy para um sistema de ordenação que permite trocar o algoritmo dinamicamente. Implemente bubble sort, selection sort e quick sort como estratégias.

```
const bubbleSort = arr => {
  const a=[...arr];
  for(let i=0;i<a.length;i++) for(let j=0;j<a.length-i-1;j++) if(a[j]>a[j+1]) [a[j],a[j+1]]=
[a[j+1],a[j]];
  return a;
};
const quickSort = arr => {
  if(arr.length<=1) return arr;
  const [piv,...rest]=arr;
  return [...quickSort(rest.filter(x=>x<=piv)),piv,...quickSort(rest.filter(x=>x>piv))];
};
const Sorter = { estrategia: bubbleSort };
Sorter.setEstrategia = fn => Sorter.estrategia=fn;
Sorter.ordenar = arr => Sorter.estrategia(arr);

console.log(Sorter.ordenar([3,1,4,1,5]));
Sorter.setEstrategia(quickSort);
console.log(Sorter.ordenar([3,1,4,1,5]));
```

■ *Strategy permite trocar algoritmos sem modificar o código que os usa — Open/Closed Principle.*

■ Strategy · padrão de design · bubble sort · quick sort · recursão

Implemente o padrão Command com histórico, suportando `execute()`, `undo()` e `redo()` para um editor de texto simples.

```
const Editor = (() => {  
  let texto = '', hist = [], pos = -1;  
  const exec = cmd => { hist.splice(pos+1); hist.push(cmd); pos++; cmd.execute(); };  
  return {  
    inserir: s => exec({  
      execute: () => { texto+=s; console.log(texto); },  
      undo: () => { texto=texto.slice(0,-s.length); },  
    }),  
    undo: () => { if(pos>=0) { hist[pos].undo(); pos--; console.log(texto); } },  
    redo: () => { if(pos<hist.length-1) { pos++; hist[pos].execute(); } },  
    texto: () => texto,  
  };  
})();  
Editor.inserir('Olá'); Editor.inserir(' Mundo');  
Editor.undo(); Editor.undo();  
Editor.redo();
```

■ *Command + histórico implementa undo/redo — base de editores de código e ferramentas criativas.*

■ Command · padrão de design · undo · redo · closure · histórico

Implemente a fase de análise do algoritmo de Huffman: construir a árvore de frequências e gerar os códigos binários para cada caractere.

```
const huffmanCodes = str => {
  const freq = [...str].reduce((m,c)=>(m.set(c,(m.get(c)||0)+1),m), new Map());
  let heap = [...freq].map(([c,f])=>({c,f,l:null,r:null}))
    .sort((a,b)=>a.f-b.f);
  while (heap.length > 1) {
    const [a,b,...rest] = heap;
    const no = {c:null,f:a.f+b.f,l:a,r:b};
    heap = [...rest,no].sort((a,b)=>a.f-b.f);
  }
  const codes={};
  const dfs=(n,code='')=>{
    if(!n.l&&!n.r){codes[n.c]=code||'0';return;}
    dfs(n.l,code+'0'); dfs(n.r,code+'1');
  };
  dfs(heap[0]);
  return codes;
};
console.log(huffmanCodes('abracadabra'));
```

■ *Huffman usa árvore binária de mínima frequência — caracteres frequentes têm códigos mais curtos.*

■ Huffman · árvore · algoritmo · compressão · Map · DFS

Implemente um async iterator que pagina resultados simulados, e use `for await...of` para consumir todas as páginas transparentemente.

```
async function* paginar(totalItens, porPagina=5) {
  let offset = 0;
  while (offset < totalItens) {
    // Simula busca assíncrona
    await new Promise(r => setTimeout(r, 50));
    const pagina = Array.from(
      {length: Math.min(porPagina, totalItens-offset)},
      (_, i) => ({id: offset+i+1, dado: `item-${offset+i+1}`})
    );
    yield pagina;
    offset += porPagina;
  }
}

(async () => {
  let total = 0;
  for await (const pagina of paginar(13, 5)) {
    total += pagina.length;
    console.log(`Página com ${pagina.length} itens`);
  }
  console.log('Total:', total);
})();
```

■ *Async generators + `for await...of` é o padrão moderno para streaming e paginação de dados.*

■ `async generator` · `for await` · `paginação` · `iterador` · `Promise`

Implemente Stack (LIFO) e Queue (FIFO) com métodos completos. Use Stack para verificar balanceamento de parênteses, e Queue para simular uma fila de atendimento.

```
class Stack {
  #items=[];
  push(v){this.#items.push(v)}
  pop(){return this.#items.pop()}
  peek(){return this.#items[this.#items.length-1]}
  isEmpty(){return!this.#items.length}
}
class Queue {
  #items=[];
  enqueue(v){this.#items.push(v)}
  dequeue(){return this.#items.shift()}
  isEmpty(){return!this.#items.length}
}
const balanceado = s => {
  const st=new Stack(), pares={'(':')','[':']','{':'}'};
  for(const c of s){
    if(c in pares) st.push(c);
    else if(Object.values(pares).includes(c)){
      if(st.isEmpty()||pares[st.pop()]!==c) return false;
    }
  }
  return st.isEmpty();
};
console.log(balanceado('{{[]}}')); // true
console.log(balanceado('{{[]}}')); // false
```

■ *Private fields (#) em classes JS (ES2022) garantem encapsulamento real — não acessíveis fora.*

■ Stack · Queue · classe · private fields · algoritmo · LIFO · FIFO

1
4
6

Programação dinâmica: mochila

Difícil · Algoritmos

Implemente o problema da mochila 0/1 (knapsack): dado um array de itens com peso e valor, e uma capacidade máxima, encontre o valor máximo possível.

```
const mochila = (itens, capacidade) => {
  const n = itens.length;
  const dp = Array.from({length:n+1}, ()=>Array(capacidade+1).fill(0));
  for (let i=1;i<=n;i++) {
    const {peso,valor} = itens[i-1];
    for (let c=0;c<=capacidade;c++) {
      dp[i][c] = dp[i-1][c];
      if (peso<=c)
        dp[i][c] = Math.max(dp[i][c], dp[i-1][c-peso]+valor);
    }
  }
  return dp[n][capacidade];
};

const itens = [{peso:2,valor:6},{peso:3,valor:10},{peso:4,valor:12}];
console.log(mochila(itens, 5)); // 16 (itens 1+2)
```

■ *Programação dinâmica preenche uma tabela de subproblemas para evitar recálculo — $O(n*W)$.*

■ programação dinâmica · mochila · knapsack · matriz · algoritmo

1
4
7

Observador reativo (mini-reactivity)

Difícil · Objetos

Implemente um sistema reativo simples usando Proxy onde propriedades observadas notificam automaticamente seus dependentes quando mudam.

```
const reativo = (obj, onChange) => new Proxy(obj, {
  set(alvo, prop, valor) {
    const anterior = alvo[prop];
    alvo[prop] = valor;
    if (anterior !== valor) onChange(prop, valor, anterior);
    return true;
  }
});

const estado = reativo(
  { contador: 0, nome: 'Ana' },
  (prop, novo, ant) => console.log(`${prop}: ${ant} → ${novo}`)
);

estado.contador++; // contador: 0 → 1
estado.nome = 'Bob'; // nome: Ana → Bob
estado.contador += 4; // contador: 1 → 5
```

■ *Proxy com setter é a base da reatividade no Vue 3 e em mini-frameworks modernos.*

■ Proxy · reatividade · observer · setter · objeto · Vue

1
4
8

Parser de Markdown simples

Difícil · Strings

Implemente um parser de Markdown que converte **negrito**, *itálico*, `código`, # Título e [link](url) para HTML.

```
const parseMarkdown = md => md
  .replace(/^# (.+)$/gm,      '<h1>$1</h1>')
  .replace(/^## (.+)$/gm,     '<h2>$1</h2>')
  .replace(/\*\*(.+?)\*\*/g,  '<strong>$1</strong>')
  .replace(/\*(.+?)\*/g,     '<em>$1</em>')
  .replace(/`(.+?)`/g,       '<code>$1</code>')
  .replace(/\[([.+?\\]\*\[([.+?\\]\*)\]/g, '<a href="$2">$1</a>')
  .replace(/^(?!<).+$/gm,    s=><p>${s}</p>`);

const md = '# Título\n**negrito** e *itálico*\n`código` e [link](https://exemplo.com)';
console.log(parseMarkdown(md));
```

■ *Parsers por regex são frágeis para Markdown completo, mas ótimos para subconjuntos controlados.*

■ regex · Markdown · HTML · replace · string · parser

1
4
9

Grafo: BFS e DFS

Difícil · Algoritmos

Dado um grafo representado como lista de adjacência, implemente BFS (largura) para encontrar menor caminho e DFS (profundidade) para detectar ciclos.

```
const bfs = (grafo, inicio, fim) => {
  const fila=[[inicio,[inicio]]], vis=new Set([inicio]);
  while(fila.length) {
    const [no,caminho]=fila.shift();
    if(no===fim) return caminho;
    for(const viz of (grafo[no]||[])) {
      if(!vis.has(viz)) { vis.add(viz); fila.push([viz,[...caminho,viz]]); }
    }
  }
  return null;
};

const grafo={A:['B','C'],B:['D'],C:['D','E'],D:['F'],E:['F'],F:[]};
console.log(bfs(grafo,'A','F')); // ['A','C','E','F'] (um caminho mais curto)
```

■ *BFS explora por níveis — garante menor número de arestas, não menor peso.*

■ grafo · BFS · DFS · algoritmo · Set · fila · caminho

1
5
0

Compilador de template

Difícil · Desafio Final

Implemente um motor de templates que suporta: interpolação `{{var}}`, condicionais `{{#if cond}}...{{/if}}` e loops `{{#each arr}}...{{/each}}`.

```
const template = (tpl, ctx) => {
  return tpl
    .replace(/{{#each (\w+)}}([\s\S]*?){{\/#each}}/g, (_, k, body) =>
      (ctx[k]||[]).map(item =>
        body.replace(/{{this}}/g, item)
          .replace(/{{(\w+)}}/g, (_, p) => item[p]??''))
        .join('')
      )
    .replace(/{{#if (\w+)}}([\s\S]*?){{\/#if}}/g, (_, k, body) =>
      ctx[k] ? body : ''
    )
    .replace(/{{(\w+)}}/g, (_, k) => ctx[k] ?? '');
};

const tpl = `Olá, {{nome}}!
{{#if admin}}Área admin{{/if}}
{{#each itens}}<li>{{this}}</li>\n{{/each}}`;

console.log(template(tpl, {nome: 'Ana', admin: true, itens: ['JS', 'CSS']}));
```

■ Este padrão é o núcleo de Handlebars, Mustache e EJS — substituições regex recursivas.

■ template · regex · replace · string · compilador · Handlebars